

PL/SQL

Variables, constantes, tableaux, Record, Contrôle du flux, boucles

• Introduction:

- **Problématique**
 - Comme en programmation, un code SQL est souvent répété.
 - Nécessité d'exécuter plusieurs requêtes pour une seule tâche.
- **Solution :**
 - Créer (comme en programmation):
 - Des fonctions
 - Des procédures
 - Déclencheurs
- **Conséquences:**
 - Allégement maxi des traitements sur le client
 - Structuration plus propre du code SQL

Introduction

- Une procédure stockée
 - compilé une fois pour toutes
 - peut être exécuté plusieurs fois. | Implique une vitesse exécution accrue
- Un déclencheur
 - est un type spécifique de procédure stockée qui **n'est pas appelé directement par un utilisateur.**
 - Lorsque le déclencheur est créé, il est défini de façon à se déclencher lorsqu'un certain type de **modification de données** est effectué dans une table.

Introduction

- Le PL/SQL (Procedural Language extensions to SQL) est un langage procédural d'ORACLE.
- L'intérêt du PL/SQL est de pouvoir dans un même traitement combiner la puissance des instructions SQL et la souplesse d'un langage procédural.
- Dans l'environnement SQL, les ordres du langage sont exécutés les uns à la suite des autres.
- Dans l'environnement PL/SQL, les ordres SQL et PL/SQL sont regroupés en blocs; un bloc ne demande qu'un seul transfert d'exécution de l'ensemble des commandes contenues dans le bloc.

Utilisation de PL/SQL

- **Le PL/SQL peut être utilisé sous 3 formes :**
 - un bloc de code (aussi appelé bloc anonyme) , exécuté comme une unique commande SQL, via un interpréteur standard (SQLplus ou iSQL*Plus)
 - un fichier de commande PL/SQL
 - un programme stocké (procédure, fonction, trigger)

Blocs

- **Un programme est structuré en blocs d'instructions de 3 types :**
 - procédures ou bloc anonymes,
 - procédures nommées,
 - fonctions nommées.
- **Un bloc peut contenir d'autres blocs.**

Structure d'un bloc

DECLARE

Déclaration des variables locales, constantes, exceptions, curseurs, modules locaux

BEGIN

Commandes exécutables: Instructions SQL et PL/SQL.
Possibilité de blocs imbriqués.

EXCEPTION

code de gestion des erreurs

**!!! Seuls BEGIN et
END sont obligatoires**

END;

/

Structure d'un bloc

- Un bloc PL/SQL peut contenir:
 - Toute instruction du LMD (SELECT, INSERT, UPDATE, DELETE)
 - Les commandes de gestion des transactions (COMMIT, ROLLBACK, SAVEPOINT)
- Les sections DECLARE et EXCEPTION sont optionnelles.
- Chaque instruction se termine par ;
- Les blocs peuvent être imbriqués
 - Les sous blocs ont la même structure que les blocs.
 - Une variable est visible dans le bloc où elle est déclarée et dans tous ses sous-blocs.
 - Si une variable est déclarée dans un premier bloc et aussi dans un sous bloc, la variable du bloc supérieur n'est plus visible dans le sous-bloc.

Affichage

- Pour afficher le contenu d'une variable, les procédures `DBMS_OUTPUT.PUT()` et `DBMS_OUTPUT.PUT_LINE()` prennent en argument **une** valeur à afficher ou **une** variable dont la valeur est à afficher.
- Par défaut, les fonctions d'affichage sont désactivées Il convient de les activer avec la commande `SET SERVEROUTPUT ON`.

- Exemple

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
c varchar2(15) := 'Bonjour!';
```

```
BEGIN DBMS_OUTPUT.PUT_LINE(c);
```

```
END;
```

```
/
```

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
c varchar2(15) := 'Bonjour !';
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT(c);
```

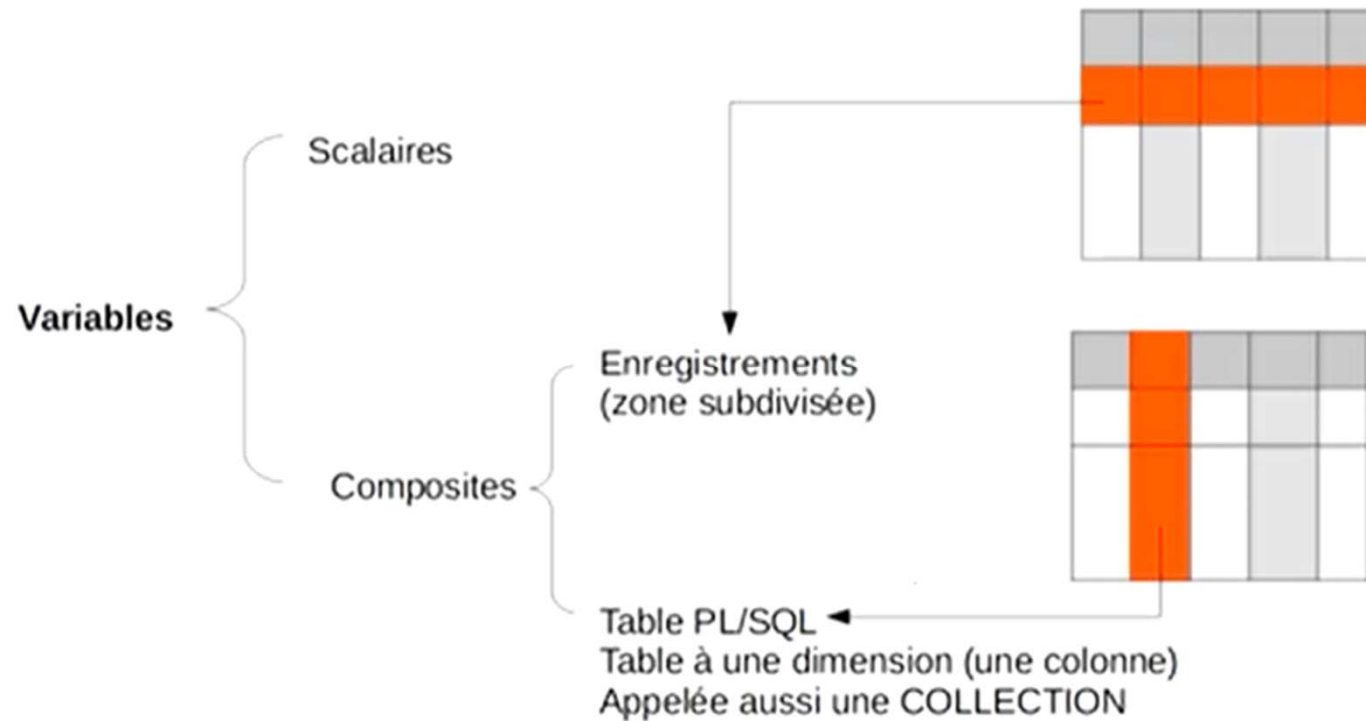
```
    DBMS_OUTPUT.NEW_LINE ;
```

```
END;
```

Structure d'un bloc PL/SQL

- **Variables**
 - Identificateurs Oracle :
 - 30 caractères au plus
 - commence par une lettre
 - peut contenir des lettres, des chiffres, _, \$ et #
 - Pas sensible à la casse
 - Doivent être déclarées avant d'être utilisées
- **Commentaires**
 - Pour un commentaire sur une ligne
 - /* Pour commentaire sur plusieurs
lignes */

Déclaration des variables



Types de variables

- **Types de variables**
 - Les types habituels correspondants aux types SQL2 ou Oracle : **integer**, **varchar**,...
 - Types composites adaptés à la récupération des colonnes et lignes des tables SQL : **%TYPE**, **%ROWTYPE**
 - Type référence : **REF**

Déclaration des variables scalaires

- **Déclaration par copie du type de donnée:**
 - La variable fait référence au même type qu'une colonne ou au même type qu'une autre variable .

Syntaxe:

Nom-de-variable nom_table.nom-colonne%type ;

ou

Nom-de-variable1 Nom-de-variable2%type ;

Exemple:

DECLARE

Nom_client client.nom%type ;

X number (10,3) ;

Y X%type ;

L'attribut %ROWTYPE

- L'attribut **% ROWTYPE** permet de déclarer une variable de même type que l'enregistrement

Syntaxe:

Nom_de_variable nom_table **%ROWTYPE** ;

Exemple:

DECLARE enrg_empl empl**%ROWTYPE** ;

- Les éléments de la structure sont identifiés par:
nom_variable . nomcolonne

Utilisation d'une variable de type Record

- Un record permet de définir des types composites

Syntaxe :

TYPE nom_record **IS RECORD**

(nom_ch 1 type ,
nom_ch2 type,
.....);

- Déclaration d'une variable de ce type: **nom_variable** **nom_record**

Exemple : **DECLARE**

TYPE enrg_empl **IS RECORD**

(nom employe. nomempl%TYPE,
salaire employe . sal %TYPE);

e enrg_empl ;

Déclaration des variables scalaires

- La partie déclarative d'un bloc PL/SQL peut comporter 3 types de déclarations:
 - Déclarations des variables, constantes,
 - Déclaration des curseurs ,
 - Déclaration des exceptions.
- 1. **Variables ou constantes :**
- **Nom-de-variable [CONSTANT] TYPE [NOT NULL] [:= | DEFAULT Expression] ;**
Expression: peut être une constante ou un calcul faisant éventuellement référence à une variable précédemment déclarée

Exemple:

```
DECLARE
```

```
Nom_duclient char (30);
```

```
X number DEFAULT 10 ;
```

```
PI constant number(7,5):=3.14159;
```

- Déclarations multiples interdites : i, j integer;

Déclaration des variables scalaires

- **Valorisation des variables : 3 possibilités:**
 - Par l'opérateur d'affectation **:=**
 - Directive **INTO** de la requête **SELECT**
 - Par le traitement d'un **curseur** dans la section **BEGIN**.
 - **Par l'opérateur d'affectation :=**
 - Nom_variable:= expression
 - **Exemple :**
X := 0;
Y := (X+5) * Y;
 - **Par la clause Select....Into**
 - **Exemple:**
DECLARE
Vref NUMBER(5,0);
Vprix produit.PU%type ;
BEGIN
SELECT REF, PU INTO Vref, Vprix
FROM Produit WHERE REF= 1 ;
dbms_output.put_line('NUMPROD='||Vref||' PU='||vprix);
END;
-

SELECT..... INTO

- Instruction **SELECT** *expr1, expr2, ...* **INTO** *var1, var2, ...*
 - Met les valeurs récupérées de la BD dans une ou plusieurs variables *var1, var2, ...*
 - **SELECT** ne doit retourner qu'une seule ligne
 - Avec Oracle il n'est pas possible d'inclure un **SELECT** sans **INTO** dans le corps d'un bloc *pl/sql*.

Déclaration des variables composites

Les collections VARRAY – Définition & Allocation

- Les types tableau peuvent être définis explicitement en utilisant la syntaxe suivante

Syntaxe :

TYPE nom_type **IS** {VARRAY | VARYING ARRAY} (size_limit) **OF** typeElements **[NOT NULL];**

- nom_type : est le nom du type tableau créé par cette instruction
- taille : est le nombre maximal d'éléments du tableau
- typeElements : est le type des éléments qui vont être stockés dans le tableau.

Exemple

TYPE numberTab **IS** VARRAY (10) **OF** NUMBER;

- Allocation d'un tableau

DECLARE

TYPE numberTab **IS** VARRAY (10) **OF** NUMBER; -- définition de type numberTab

tab numberTab; -- déclaration de tableau tab

BEGIN

tab := numberTab (); -- allocation de l'espace mémoire avec constructeur de type

/* utilisation du tableau */

END;

Déclaration des variables composites

Les collections VARRAY – Dimensionnement

DECLARE

```
TYPE numberTab IS VARRAY (10) OF NUMBER;
```

```
tab numberTab ;
```

BEGIN

```
tab := numberTab ( ) ;
```

```
tab.EXTEND( 4 ) ;
```

```
/* utilisation du tableau */
```

END;

/

- Dans cet exemple, tab.EXTEND(4) permet par la suite d'utiliser les éléments du tableau t(1), t(2), t(3) et t(4). **Il n'est pas possible "d'étendre" un tableau à une taille supérieure à celle spécifiée lors de la création du type tableau associé.**
- On peut aussi utiliser le constructeur avec des valeurs d'initialisations.
tab := numberTab (1,2,3,4) ;

Déclaration des variables composites

Les collections TABLE

- Les types tableau peuvent aussi être définis explicitement par la syntaxe suivante :
TYPE nom_type **IS TABLE OF** type_des_valeurs **INDEX BY** type_des_clés ;
 - **nom_type** : est le nom du type tableau créé par cette instruction,
 - **type_des_valeurs** : est le type des éléments qui vont être stockés dans le tableau,
 - **type_des_clés** est **BINARY_INTEGER** ou **VARCHAR(10)** par exemple. Ce sont les clés du tableau associatif, elles doivent être uniques !
 - Les types admis doivent être numériques (**BINARY_INTEGER** ou **PLS_INTEGER**) ou alphabétique (**VARCHAR(longueur)**, **STRING**, **LONG** ou **VARCHAR2(longueur à préciser obligatoirement)**).
 - Si on veut utiliser d'autre type (dates), il faudra faire une conversion.

Les collections TABLE

Déclaration d'un tableau : `nom_Tableau nom_Type ;`

Exemple

`declare`

`-- collection de type table sans index by`

`TYPE TYP_TAB is table of varchar2(100) ;`

`-- collection de type table avec index by`

`TYPE TYP_TAB_IND is table of number index by binary_integer ;`

`tab1 TYP_TAB ;`

`tab2 TYP_TAB_IND ;`

`Begin`

`tab1 := TYP_TAB('Lundi','Mardi','Mercredi','Jeudi' ) ;`

`for i in 1..10 loop`

`tab2(i):= i ;`

`end loop ;`

`End;`

Les collections TABLE

- -- table de multiplication par 8 et par 9...
- declare
- type tablemul is record (par8 number, par9 number);
- type tableentiers is table of tablemul index by binary_integer;
- ti tableentiers;
- i number;
- begin
- for i in 1..10 loop
- ti(i).par9 := i*9;
- ti(i).par8:= i*8;
- dbms_output.put_line (i||'*8='||ti(i).par8||' '||i||'*9='||ti(i).par9);
- end loop;
- end;



1*8=8	1*9=9
2*8=16	2*9=18
3*8=24	3*9=27
4*8=32	4*9=36
5*8=40	5*9=45
6*8=48	6*9=54
7*8=56	7*9=63
8*8=64	8*9=72
9*8=72	9*9=81
10*8=80	10*9=90

Méthodes intégrées pour les collections

- **Un constructeur de collection** (constructeur) est une fonction définie par le système avec le même nom qu'un type de collection, qui renvoie une collection de ce type. La syntaxe d'un appel de constructeur est:
`collection_type ( [ value [, value ]... ] )`
- **EXISTS** - Utilisé pour déterminer si un élément spécifique d'une collection existe. EXISTS est utilisé avec des tables imbriquées.
`Tab.exists(valeur)`
- **COUNT** - Retourne le nombre d'éléments actuellement contenus dans une collection sans inclure les valeurs **NULL**. Pour varrays count est égal à **LAST**. Pour les tables imbriquées, **COUNT** et **LAST** peuvent être différents en raison de valeurs supprimées dans les sites de données interstitielles dans la table imbriquée.
`Tab.count`
- **LIMIT** - Utilisé pour **VARRAYS** pour déterminer le nombre maximum de valeurs autorisées. Si **LIMIT** est utilisé sur une table imbriquée, elle renverra une valeur nulle.
`Tab.LIMIT`
- **FIRST** et **LAST** - Renvoie les plus petits et les plus grands numéros d'index pour la collection référencée. Naturellement, ils renvoient null si la collection est vide
`Tab.First,      Tab.Last`

Méthodes intégrées pour les collections

- **PRIOR** et **NEXT** - Renvoie la valeur précédente ou suivante en fonction de la valeur d'entrée de l'index de collection. **PRIOR** et **NEXT** ignorent les instances supprimées d'une collection.
`indice_suivant:=tab.next(indice);`
- **EXTEND** - Ajoute des instances à une collection. **EXTEND** a trois formes, **EXTEND**, qui ajoute une instance nulle, **EXTEND (n)** qui ajoute "n" instances **NULL** et **EXTEND (n, m)** qui ajoute n copies de l'instance "m" à la collection. Pour les formes de collections spécifiées non nulles, les formes un et deux ne peuvent pas être utilisées.
- **DELETE** - **DELETE** supprime les éléments spécifiés d'une table imbriquée ou d'une table
`tab.Delete(indice)`
- **VARRAY**. **DELETE** spécifié sans argument supprime toutes les instances d'une collection. Pour les tables imbriquées, seul **DELETE (n)** supprime la n^{ième} instance et **DELETE (n, m)** supprime la n^{ième} à travers les instances mth de la table imbriquée qui se rapportent à la fiche spécifiée.

Exemple

```
i := tab.FIRST;  
WHILE i IS NOT NULL LOOP  
  traitement  
  i := tab.NEXT(i);  
END LOOP;
```

-
- Cette boucle s'arrête au dernier élément parce que NEXT(i) renvoie NULL au dernier i.

Collections & Exception

- Les méthodes de collecte peuvent générer les exceptions suivantes:
 - **COLLECTION_IS_NULL** - générer lorsque la collection référencée est nulle.
 - **NO_DATA_FOUND** - L'indice indique une instance nulle de la collection.
 - **SUBSCRIPT_BEYOND_COUNT** - L'indice spécifié dépasse le nombre d'instances de la collection.
 - **SUBSCRIPT_OUTSIDE_LIMIT** - L'indice spécifié est en dehors de la plage autorisée (généralement reçue des références VARRAY)
 - **VALUE_ERROR** - L'indice est nul ou n'est pas un nombre entier.

Comment parcourir un tableau associatif (table)

- Accées direct à une valeur en utilisant la clé
 nom_Tableau(clé)
- FIRST et LAST
 - Un tableau associatif (comme toutes les autres collections en PL/SQL) a les propriétés FIRST et LAST auxquelles on accède en faisant **nom_tableau.FIRST** et **nom_tableau.LAST**.
 - FIRST renvoie **la première clé** issue d'un tri numérique ou alphabétique.
 - Pour parcourir le tableau, on peut donc faire une boucle comme ceci :
 FOR i IN nom.FIRST..nom.LAST LOOP
- PRIOR et NEXT

```
    i := nom.FIRST;
    WHILE i IS NOT NULL LOOP
        ...
        i := nom.NEXT(i);
    END LOOP;
```

 - Cette boucle s'arrête au dernier élément parce que NEXT(i) renvoie au dernier i.

Opérateurs PL/SQL

- Opérateurs logiques NOT, AND, OR
- Opérateurs arithmétiques + - * / ** (puissance)
- Opérateurs de comparaison =, >, !=, >, <=, >=, IS NULL, LIKE, BETWEEN
- Opérateurs de chaîne de caractères || (Concaténation)

Utiliser des parenthèses pour simplifier l'écriture et la lecture des expressions

Structures de contrôles

- **Structure alternative**
 - IF THEN.... END IF ;
 - IF THEN.... ELSE ... END IF ;
- **Itérations**
 - LOOP ..
 - FOR i.....
 - WHILE
- **Branchement séquentiels**
 - GOTO
 - NULL

- **IF THEN**
 IF condition THEN
 sequence_insts;
 END IF ;
 - **IF THEN ELSE**
 IF condition THEN
 sequence_insts1;
 ELSE
 sequence_insts2;
 END IF ;
 - **IF THEN ELSIF**
 IF condition1 THEN
 sequence_insts1;
 ELSIF condition2 THEN
 sequence_insts2;
 ELSE
 sequence_insts3;
 END IF ;
-

Structures alternatives

- **IF THEN**
 IF condition THEN
 sequence_insts;
 END IF ;
- **IF THEN ELSE**
 IF condition THEN
 sequence_insts1;
 ELSE
 sequence_insts2;
 END IF ;

Itérations

- LOOP-EXIT-WHEN-END

LOOP

.....

EXIT WHEN CONDITION ou EXIT

.....

END LOOP

- Exemple:

i:= 1 ;

LOOP

i:= i+1 ;

EXIT WHEN i >= 10 ;

....

END LOOP

Itérations

- WHILE-LOOP-END
WHILE condition LOOP

.....
END LOOP
- Exemple:
WHILE total<= 10000 LOOP

.....
SELECT salaire INTO S FROM employe WHERE.....

....
total:= total + S ;
END LOOP

Itérations

- FOR-IN-LOOP-END

```
FOR compteur IN [ REVERSE] inf..sup LOOP
```

```
.....
```

```
END LOOP
```

- Exemple:

```
FOR I IN 1..10 LOOP
```

```
    J:= J* 3;
```

```
    INSERT INTO T VALUES ( I, J );
```

```
END LOOP
```

```
declare  
    i int;  
begin  
    FOR i IN REVERSE 1..10 LOOP  
        DBMS_OUTPUT.PUT_LINE(i);  
    END LOOP;  
end;
```

Null

- Signifie «ne rien faire » , passer à l'instruction suivante

- Exemple:

```
IF I>=10 THEN
```

```
    NULL ;
```

```
ELSE
```

```
    INSERT INTO T VALUES ( 'inférieur à 10',I );
```

```
END IF ;
```

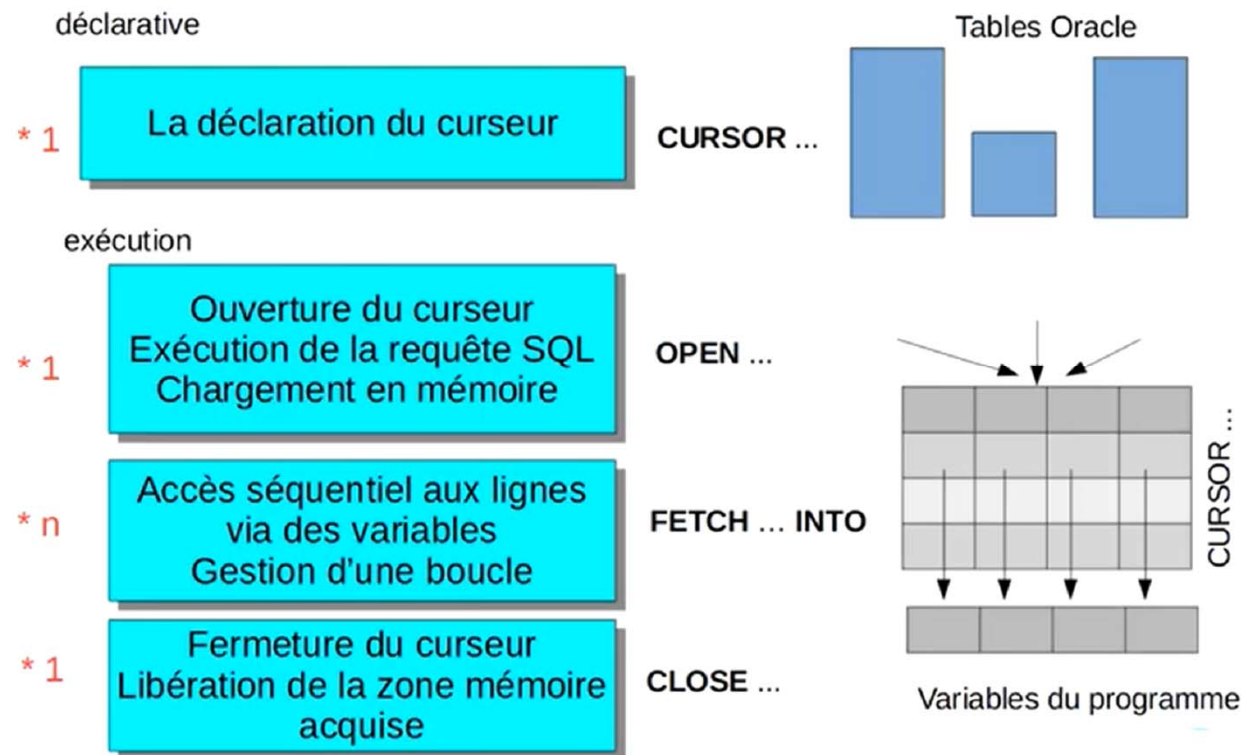

Les curseurs

- Définition d'un curseur.
- Déclaration d'un curseur.
- Utilisation d'un curseur.

Curseur - Définition

- PL/SQL utilise des curseurs pour tous les accès à des informations de la base de données.
- Un curseur est pointeurs associés au résultat d'une requête.
- Deux types de curseurs:
 - **Implicite**: créés automatiquement par Oracle pour chaque ordre SQL (**select, update, delete, insert**) .
 - **Explicite**: crée par le programmeur pour pouvoir traiter le résultat de requêtes retournant **plus** d'un tuple.

Etapes de gestion d'un curseur

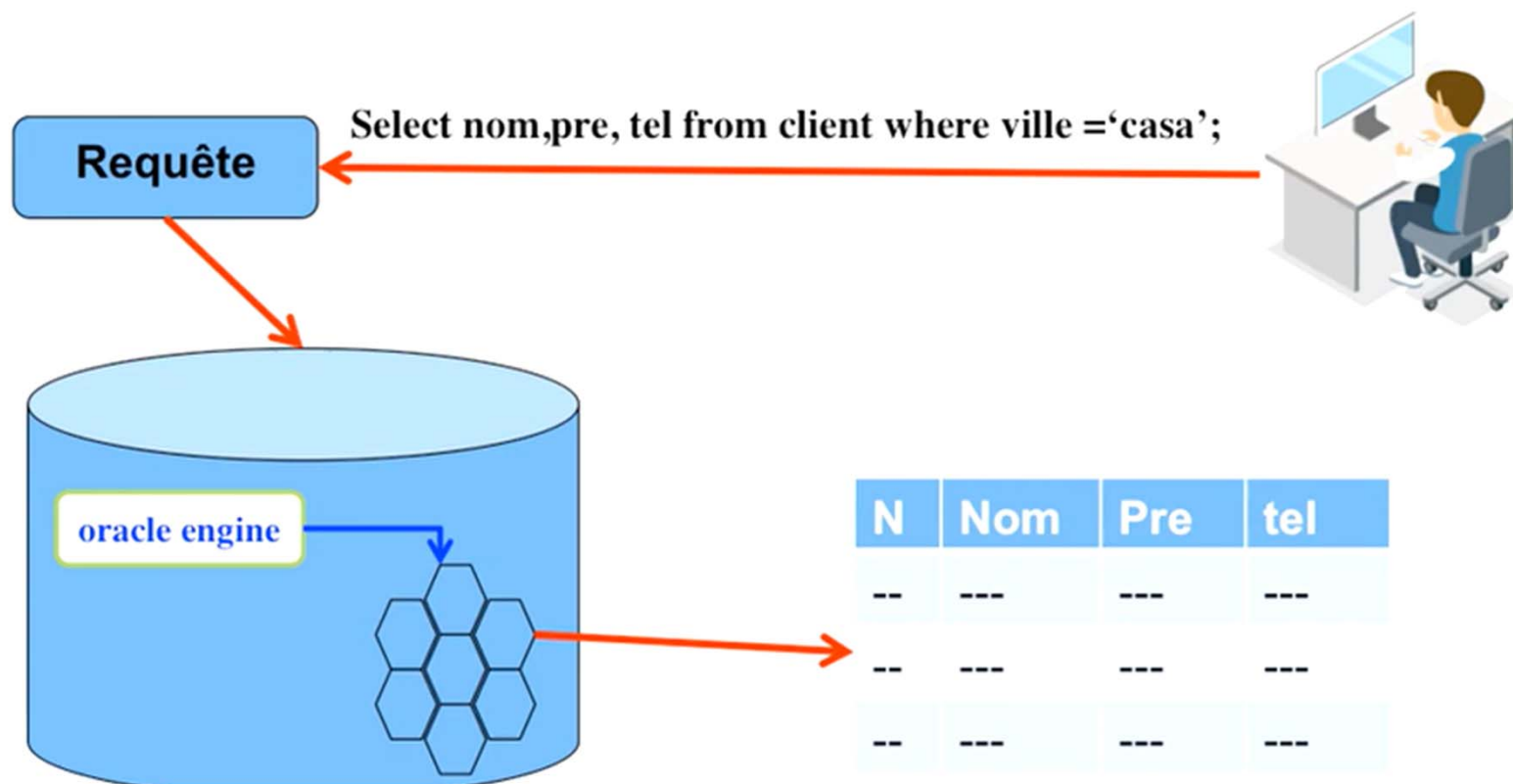


Curseurs

- **Tous les curseurs ont des attributs que l'utilisateur peut utiliser**
 - **%ROWCOUNT** : nombre de lignes traitées par le curseur
 - **%FOUND** : vrai si au moins une ligne a été traitée par la requête ou le dernier fetch
 - **%NOTFOUND** : vrai si aucune ligne n'a été traitée par la requête ou le dernier fetch
 - **%ISOPEN** : vrai si le curseur est ouvert (utile seulement pour les curseurs explicites)

Les curseurs

Un curseur est une variable qui pointe vers le résultat d'une requête SQL. La déclaration du curseur est liée au texte de la requête.



Curseurs implicite

- Les curseurs implicites sont tous nommés SQL

DECLARE

nb_lignes integer;

BEGIN

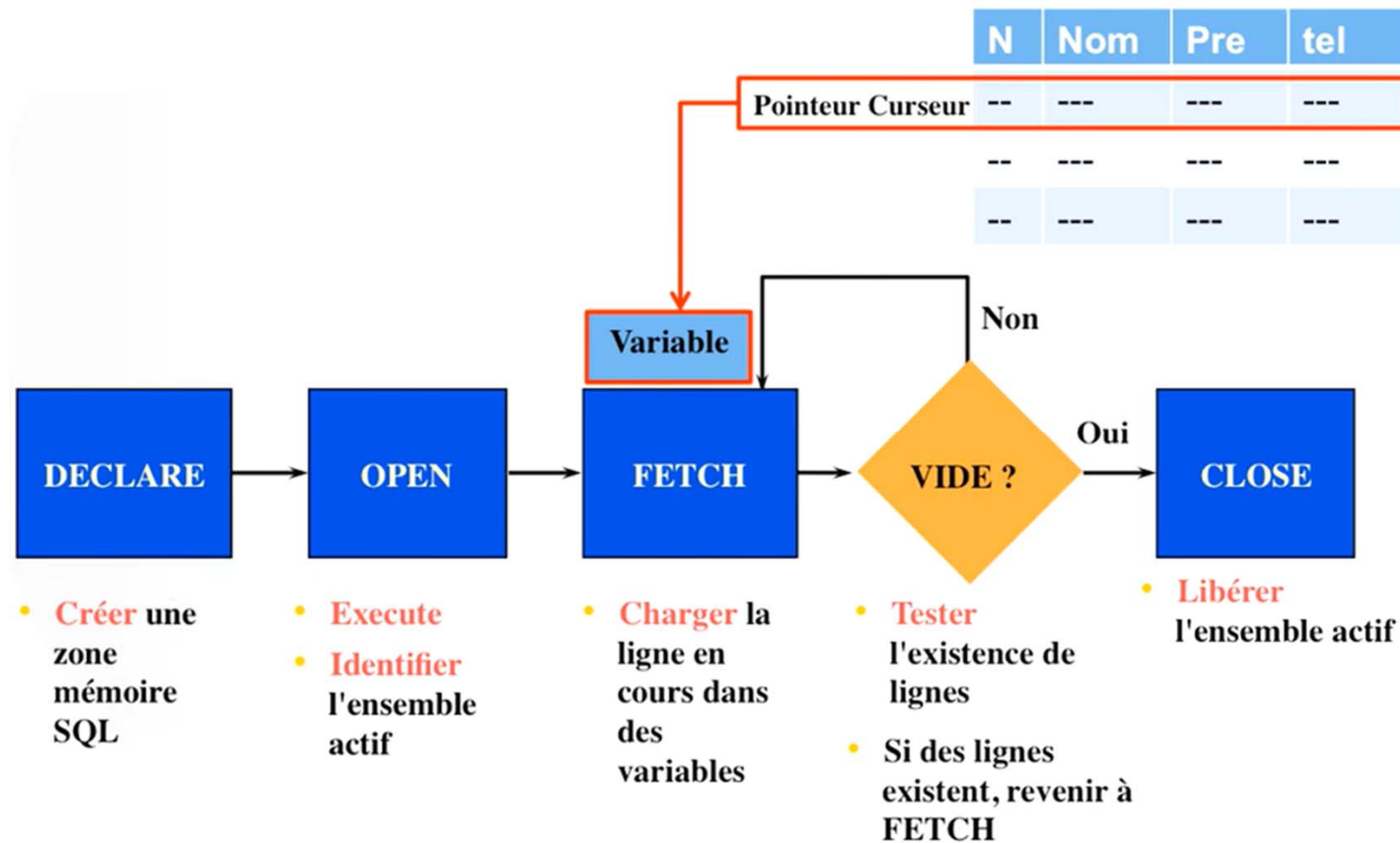
delete from empl where dept = 10;

nb_lignes := SQL%ROWCOUNT;

...

Les curseurs explicites

- **Les curseurs explicites**
 - sont utilisés pour traiter les select qui renvoient plusieurs lignes,
 - doivent être déclarés
- Le code doit être utiliser explicitement avec les ordres **OPEN**, **FETCH** et **CLOSE**
- Le plus souvent on les utilise dans une boucle dont on sort quand l'attribut **NOTFOUND** du curseur est vrai.



Curseurs explicite - déclaration

Syntaxe

- Déclaration d'un curseur **sans paramètres** :

DECLARE

CURSOR <nom_curseur> **IS** Requete_SELECT

Exemple: Déclarer un curseur qui permet de lister les employés de Kénitra

DECLARE

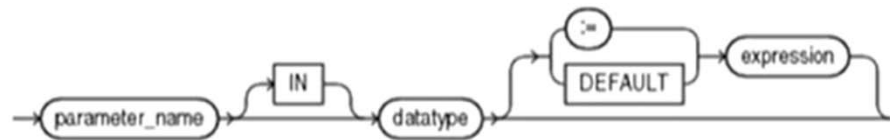
v_ville employe.ville%**type** := 'Kénitra' **--variable**

CURSOR employe_kenitra **IS** **--curseur**

SELECT numemp, nomemp **FROM** employe **WHERE** ville=
v_ville;

Curseur explicite - déclaration

- Déclaration du curseur **avec paramètres**:
CURSOR <nom_curseur> (para1, para2,.....) **IS** Requete_SELECT ;



- On doit fermer le curseur entre chaque utilisation de paramètres différents (sauf si on utilise « for » qui ferme automatiquement le curseur)

Exemple:

DECLARE

CURSOR employe (v **IN** VARCHAR2) **IS**

SELECT numemp, nomemp **FROM** employe **WHERE** ville= v;

Curseur Ouverture

- L'ouverture du curseur se fait dans la section BEGIN du bloc par:
`OPEN nom_curseur [(para1, para2 ...)]`

Exemple:

DECLARE

CURSOR employe_rabat **IS**

SELECT numemp, nomemp **FROM** employe **WHERE** ville= '
Rabat';

BEGIN

OPEN employe_rabat;

.....

.....

END;

Si aucune ligne n'est lue
⇒ OPEN ne déclenche pas d'exception
⇒ Le programme teste l'état du curseurs
après le FETCH

Curseur – Accès aux données

- L'accès aux données se fait par la clause: **FETCH INTO**

Syntaxe:

FETCH nom_curseur **INTO** < var1, var2,.....>

- Le **FETCH** permet de récupérer un tuple de l'ensemble réponse associé au curseur et stocker les valeurs dans des variables.
- Pour traiter n lignes, il faut une boucle.

Fermeture d'un Curseur

- La fermeture du curseur se fait dans la section **BEGIN** du bloc par:
CLOSE nom_curseur

- Exemple

```
DECLARE
```

```
CURSOR employe_rabat IS
```

```
SELECT numemp, nomemp FROM employe WHERE ville= 'Rabat';
```

```
BEGIN
```

```
OPEN employe_rabat
```

```
.....
```

```
CLOSE employe_rabat;
```

```
END;
```

Curseurs - Exemple

Un curseur qui récupère les employés qui sont de Rabat et on va les .
traiter Le traitement qu' on va faire , on va vérifier le salaire de chaque
employer, si le salaire > 3000 on va le stocker dans une table

```
CREATE TABLE resultat ( nom varchar2, sal number )  
DECLARE  
    CURSOR employe_rabat IS  
    SELECT numemp, sal FROM employe WHERE ville= ' Rabat' ;  
    nom employe.nomempl%TYPE ;  
    salaire employe.sal %TYPE ;  
BEGIN  
    OPEN employe_rabat ;  
    FETCH employe_rabat INTO nom, salaire ;  
    WHILE employe_rabat%found LOOP  
        IF salaire >3000 THEN  
            INSERT INTO resultat VALUES ( nom, salaire) ;  
        END IF ;  
        FETCH employe_rabat INTO nom, salaire ;  
    END LOOP ;  
    CLOSE employe_rabat ;  
END ;
```

Curseurs explicite avec paramètres - Exemple

```
declare
    CURSOR cur_employe( v_ville varchar2) IS
    SELECT nomemp, sal FROM employe WHERE ville= v_ville ;
    v_nomemp  employe.nomemploye%type ;
    v_sal  employe.sal%type ;
begin
    open cur_employe('Kénitra');
    fetch cur_employe into v_nomemp, v_sal;
    while (cur_employe%found) loop
        DBMS_output.put_line(v_nomemp||' '||v_sal);
        fetch cur_employe into v_nomemp, v_sal;
    end loop;
end;
```

Curseurs explicite avec paramètres - Exemple

- **FOR LOOP** remplace **OPEN**, **FETCH** et **CLOSE**.
- Lorsque le curseur est invoqué, **un enregistrement** est automatiquement créé avec les mêmes éléments de données que ceux définies dans l'instruction select.

declare

```
CURSOR cur_employe( v_ville varchar2) IS  
SELECT nomemp, sal FROM employe WHERE ville= v_ville ;
```

begin

```
FOR enrg_e IN cur_employe('rabat') loop  
    /* traitement*/
```

```
END LOOP;
```

```
FOR enrg_e IN cur_employe('kenitra') loop  
    /* traitement*/
```

```
END LOOP;
```

end;

Curseur - Exemple

- Donner un programme PL/SQL utilisant un curseur pour calculer la somme des salaires des employés

```
DECLARE
    CURSOR salaires IS
        select sal from emp where dept = 10;
    salaire numeric(8, 2);
    total numeric(10, 2) := 0;
BEGIN
    open salaires;
    loop
        fetch salaires into salaire;
        exit when salaires%notfound;
        if salaire is not null then
            total := total + salaire; Attention !
        end if;
    end loop;
    close salaires; -- Ne pas oublier
    DBMS_OUTPUT.put_line(total);
END;
```

Type <<rowtype>> associé à un curseur

- On peut déclarer un type « rowtype » associé à un curseur
Nom_enregistrement nom_cursuer%rowtype ;

Exemple :

```
declare
  cursor c is
    select matr, nome, sal from emp;
    employe c%ROWTYPE;
begin
  open c;
  fetch c into employe;
  if employe.sal is null then ...
```

Boucle FOR pour un curseur

- For permet de simplifier la programmation car elle évite d'utiliser explicitement les instructions `open`, `fetch`, `close`.
- En plus elle déclare implicitement une variable de type «row» associée au curseur

Exemple

`declare`

```
    cursor c is select dept, nome from emp  
    where dept = 10;
```

`begin`

```
    FOR employe IN c LOOP
```

```
        dbms_output.put_line(employe.nome);
```

```
    END LOOP;
```

`end;`

Variable de type
c%rowtype



Les attributs du curseurs explicite

Attributs	Informations obtenues
%ROWCOUNT	Nombre de lignes concernées par la dernière requête
%FOUND	TRUE si la dernière requête concernait une ou plusieurs lignes
%NOTFOUND	TRUE si la dernière requête ne concernait aucune ligne
%ISOPEN	TRUE si le curseur est ouvert

Format d'utilisation

Nom_curseur% ATTRIBUT

Curseur modifiable

- Un curseur modifiable est un curseur qui peut être utilisé pour modifier le contenu d'une table
- Un curseur qui comprend plus d'une table dans sa définition ne permet pas la modification des tables de BD.
- Seuls les curseurs définis sur une seule table sans fonction d'agrégation et de regroupement peuvent être utilisés dans une MAJ : delete, update.
- FOR UPDATE [OF col1, col2,...]. Cette clause bloque toute la ligne ou seulement les colonnes spécifiées
- Les autres transactions ne pourront modifier les valeurs tant que le curseur n'aura pas quitté cette ligne
- Pour désigner la ligne courante à modifier on utilise la clause CURRENT OF nom_curseur.

Curseur modifiable - Exemple

```
DECLARE
  Cursor C1 is
    select nomemp, sal from employe for update of sal;
  resC1 C1%rowtype;
BEGIN
  Open C1;
  Fetch C1 into resC1;
  While C1%found Loop
    If resC1.sal < 1500 then
      update employe
      set sal = sal * 1.1
      where current of c1;
    end if;
    Fetch C1 into resC1;
  end loop;
  close C1;
END;/
```

PROCEDURES & FONCTIONS

Types de blocs PL/SQL

<pre>[DECLARE] BEGIN -- phrases [EXCEPTION] END;</pre>	<pre>PROCEDURE name IS [Déclaration_variables_locales] BEGIN -- phrases [EXCEPTION] END;</pre>	<pre>FUNCTION name IS RETURN datatype [Déclaration_variables_locales] BEGIN -- phrases [EXCEPTION] END;</pre>
Bloc anonyme Imbriqué dans un programme	Procédure Effectue un traitement Peut recevoir des paramètres	Fonction Effectue un calcul et donne un résultat Peut recevoir des paramètres

Procédures et fonctions

- Une procédure/fonction stockée est composée d'instructions compilées et enregistrées dans la BD. Elle est activée par des événements ou des applications. Elle comporte, outre des ordres SQL, des instructions de langage PL/SQL (branchement conditionnel, instructions de répétition, affectations,...).
- L'intérêt d'une procédure stockée est :
 - 1) D'alléger les échanges entre client et serveur de BD en stockant au niveau du serveur les procédures régulièrement utilisées.
 - 2) D'optimiser les requêtes au moment de la compilation des procédures plutôt qu'à l'exécution.
 - 3) De renforcer la sécurité : on peut donner l'autorisation à un utilisateur d'utiliser une procédure stockée sans lui donner les droits directement sur les tables qu'elle utilise.

Procédures

- On définit une procédure de la sorte
`CREATE [OR REPLACE] PROCEDURE nom_procedure`
`[(liste_argument1 )] {IS|AS}`

```
[ -- déclaration des variables, exceptions, procédures,...locales]
BEGIN
    -- Instructions PL/SQL
[exception
    -- gestion des exceptions
]
END [nom_procedure];
```

Syntaxe pour la création de procédures

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(parameter1 [mode1] datatype1,
  parameter2 [mode2] datatype2,
  . . .)]
IS
PL/SQL Block;
```

- L'option **REPLACE** indique que, si la procédure existe, elle sera supprimée et remplacée par la nouvelle version créée avec l'instruction
- Le bloc PL/SQL commence par **BEGIN** ou par la **déclaration** de variables locales et se termine par **END** ou par **END *procedure_name***

```
CREATE OR REPLACE PROCEDURE add_numbers
( a IN NUMBER,
  b IN NUMBER,
  sum OUT NUMBER )AS
BEGIN
    sum := a + b;
END;
```

```
DECLARE
    result NUMBER;
BEGIN
    add_numbers(5, 10, result);
    DBMS_OUTPUT.PUT_LINE('La somme est : ' ||
    result);
END;
```

Dans cet exemple :

- a et b sont des paramètres formels de type IN.
- sum est un paramètre formel de type OUT.
- Lors de l'appel de la procédure, 5 et 10 sont des paramètres réels passés à a et b, respectivement

Paramètres Formels / Réels

- Les paramètres **formels** sont des variables **déclarées** dans la liste de paramètres d'une spécification de sous-programme

Exemple :

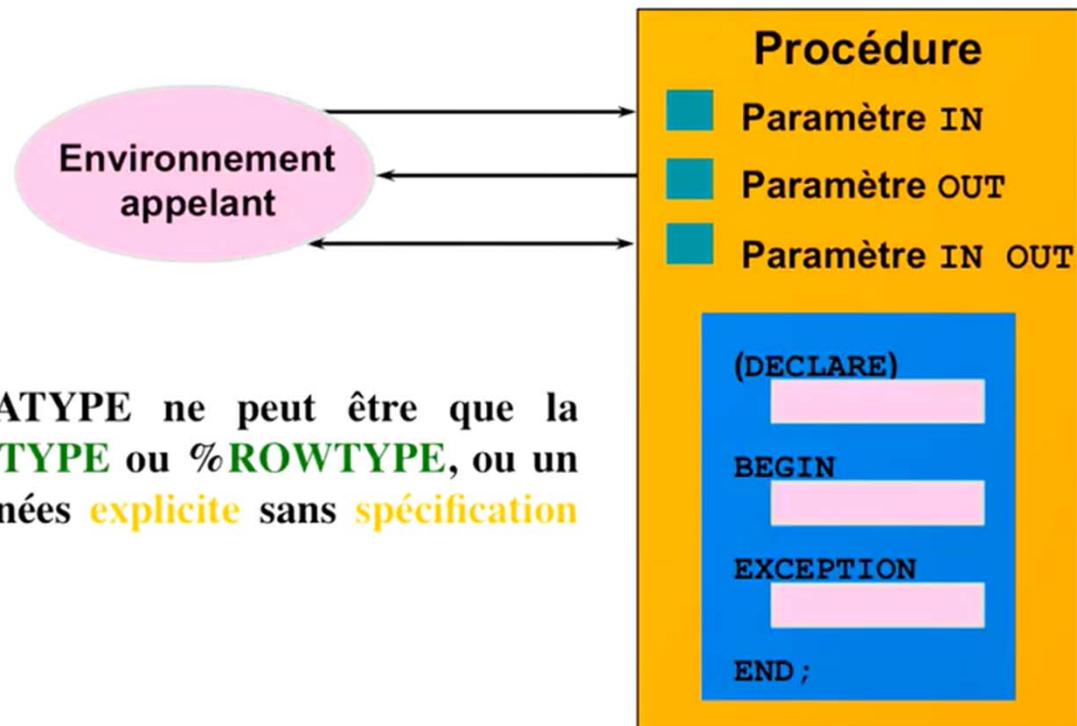
```
CREATE PROCEDURE Moy_sal ( Emp_id NUMBER, Emp_sal NUMBER)
....
END Moy_sal;
```

- Les paramètres réels sont **des variables ou des expressions référencées** dans la liste de paramètres d'un appel de sous-programme

Exemple :

```
Moy_sal(v_id, 2000)
```

Modes des paramètres des procédures



RQ : DATATYPE ne peut être que la définition **%TYPE** ou **%ROWTYPE**, ou un type de données **explicite** sans **spécification de taille**.

Créer des procédures avec des paramètres

IN	OUT	IN OUT
Mode par défaut	Doit être indiqué	Doit être indiqué
La valeur est transmise au sous-programme	Est renvoyé à l'environnement appelant	Est transmis à un sous-programme ; est renvoyé à l'environnement appelant
Le paramètre formel se comporte en constante	Variable non initialisée	Variable initialisée
Le paramètre réel peut être un littéral, une expression, une constante ou une variable initialisée	Doit être une variable	Doit être une variable

Les procédures

Exemples de paramètres IN:

```
CREATE OR REPLACE PROCEDURE modifier_salaire
  (Emp_id IN employees.employee_id%TYPE)
IS
BEGIN
  UPDATE employees
  SET    salaire = salaire * 1.10
  WHERE  employee_id = Emp_id;
END modifier_salaire;
```

Les procédures

Exemples de paramètres OUT

Environnement appelant

Procédure QUERY_EMP



Exemples de paramètres OUT

query_emp.sql

```
CREATE OR REPLACE PROCEDURE query_emp
(p_id      IN   employees.employee_id%TYPE,
p_nom      OUT  employees.nom%TYPE,
p_salaire  OUT  employees.salaire%TYPE,
p_comm     OUT  employees.commission%TYPE)
IS
BEGIN
    SELECT    nom, salaire, commission
    INTO      p_nom, p_salaire, p_comm
    FROM      employees
    WHERE     employee_id = p_id;
END query_emp;
```

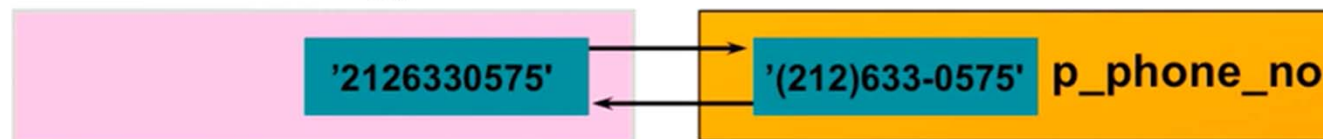
Visualiser des paramètres OUT

Déclarer les variables, exécuter la procédure QUERY_EMP, puis imprimer la valeur de la variable globale G_NAME

```
VARIABLE g_name      VARCHAR2 (25) ;  
VARIABLE g_sal       NUMBER;  
VARIABLE g_comm      NUMBER;  
  
EXECUTE query_emp(171, :g_name, :g_sal, :g_comm);  
  
PRINT g_name;
```

Exemple paramètre IN OUT

Environnement appelant



```
CREATE OR REPLACE PROCEDURE format_phone
  (p_phone_no IN OUT VARCHAR2)
IS
BEGIN
  p_phone_no := '(' || SUBSTR(p_phone_no,1,3) ||
                ')' || SUBSTR(p_phone_no,4,3) ||
                '-' || SUBSTR(p_phone_no,7,4);
END format_phone;
```

Visualiser des paramètres IN OUT

```
VARIABLE phone VARCHAR2(15)

EXECUTE :phone := '212668448477';

EXECUTE format_phone (:phone)
PRINT phone
```

Résultat :

PHONE
(212) 668-448477

Déclarer des sous-programmes

```
CREATE OR REPLACE PROCEDURE leave_emp2
  (p_id IN employees.employee_id%TYPE)
IS
  PROCEDURE log_exec
  IS
  BEGIN
    INSERT INTO log_table (user_id, log_date)
    VALUES (USER, SYSDATE);
  END log_exec;
BEGIN
  DELETE FROM employees
  WHERE employee_id = p_id;
  log_exec;
END leave_emp2;
/
```

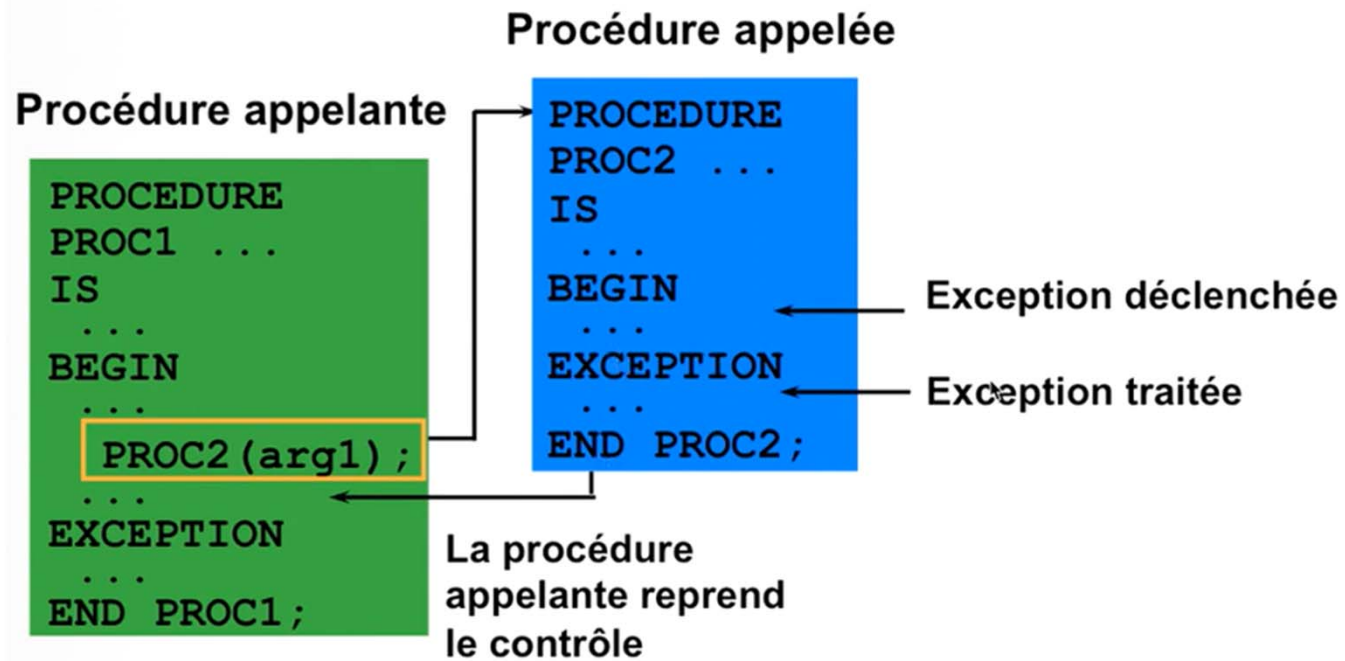
Appeler une procédure depuis un bloc PL/SQL anonyme

```
DECLARE
    v_id NUMBER := 163;
BEGIN
    Modifier_salaire(v_id); --Appel procedure
    COMMIT;
    ...
END;
```

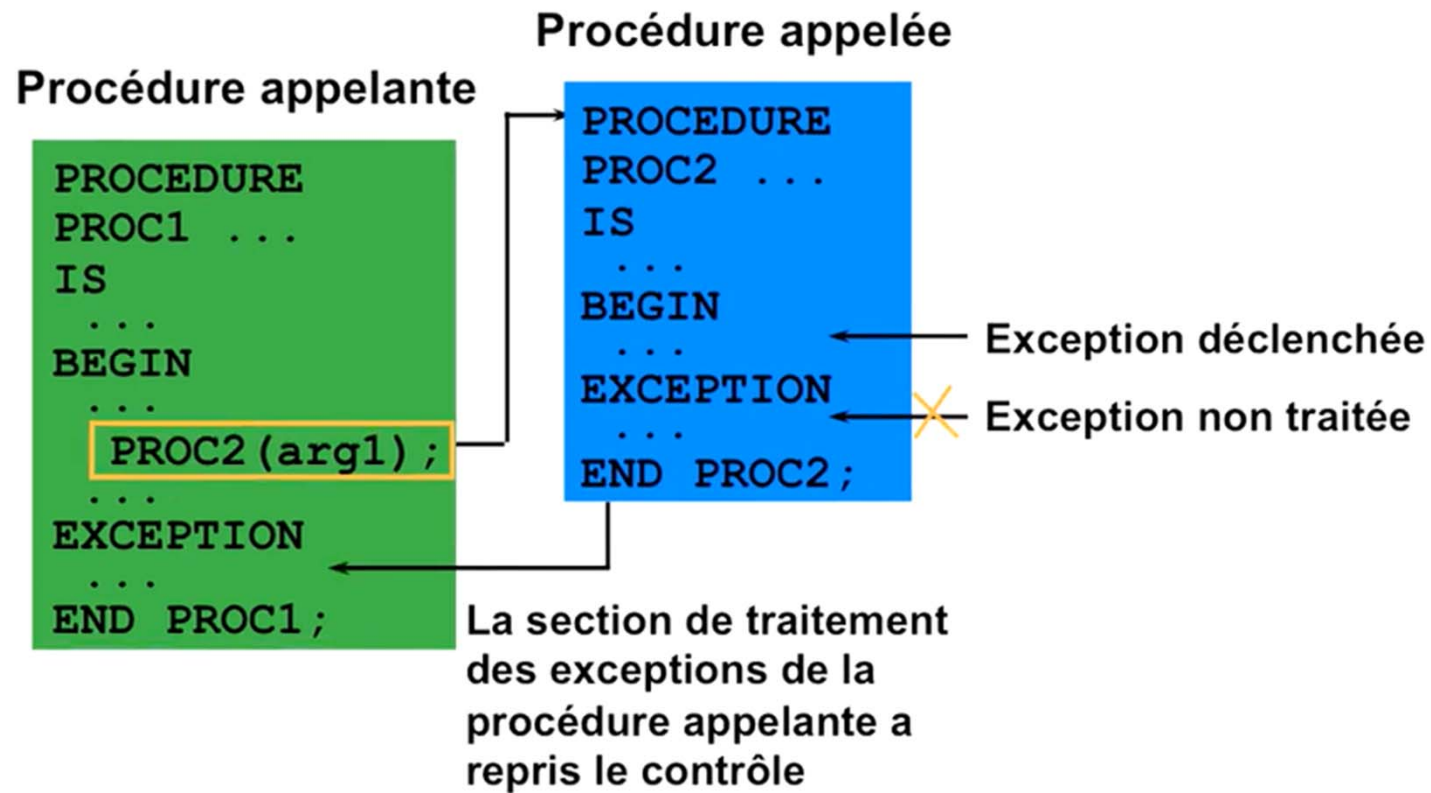
Appeler une procédure depuis une autre procédure

```
CREATE OR REPLACE PROCEDURE process_emps
IS
    CURSOR emp_cursor IS
        SELECT employee_id FROM employees;
BEGIN
    FOR emp_rec IN emp_cursor
    LOOP
        Modifier_salaire(emp_rec.employee_id);
    END LOOP;
    COMMIT;
END process_emps;
/
```

Exceptions traitées



Exceptions non traitées



Supprimer une procédure dans la base de données

Syntaxe:

```
DROP PROCEDURE procedure_name
```

Exemple :

```
DROP PROCEDURE Modifier_salaire;
```

Synthèse

- ❖ **Une procédure est un sous-programme** qui exécute une action
- ❖ Vous pouvez créer des procédures en utilisant la commande **CREATE PROCEDURE**
- ❖ Vous pouvez **compiler et enregistrer** une procédure dans la base de données
- ❖ Il existe trois **modes de paramètre** : IN, OUT et IN OUT
- ❖ Les **procédures peuvent être appelées** à partir de n'importe quel outil ou langage prenant en charge le langage PL/SQL
- ❖ Vous devez être conscient de l'impact des **exceptions traitées et non traitées** sur les transactions et les procédures appelantes
- ❖ Vous pouvez **supprimer des procédures de la base de données** en utilisant la commande **DROP PROCEDURE**

Optimisation des procédures

- Si la base de données évolue, il faut recompiler les procédures existantes pour qu'elles tiennent compte de ces modifications.
- La commande est la suivante:
`ALTER { FUNCTION | PROCEDURE } nom COMPILE`
- Exemple :
`ALTER PROCEDURE augmentation_prix COMPILE;`

Fonctions

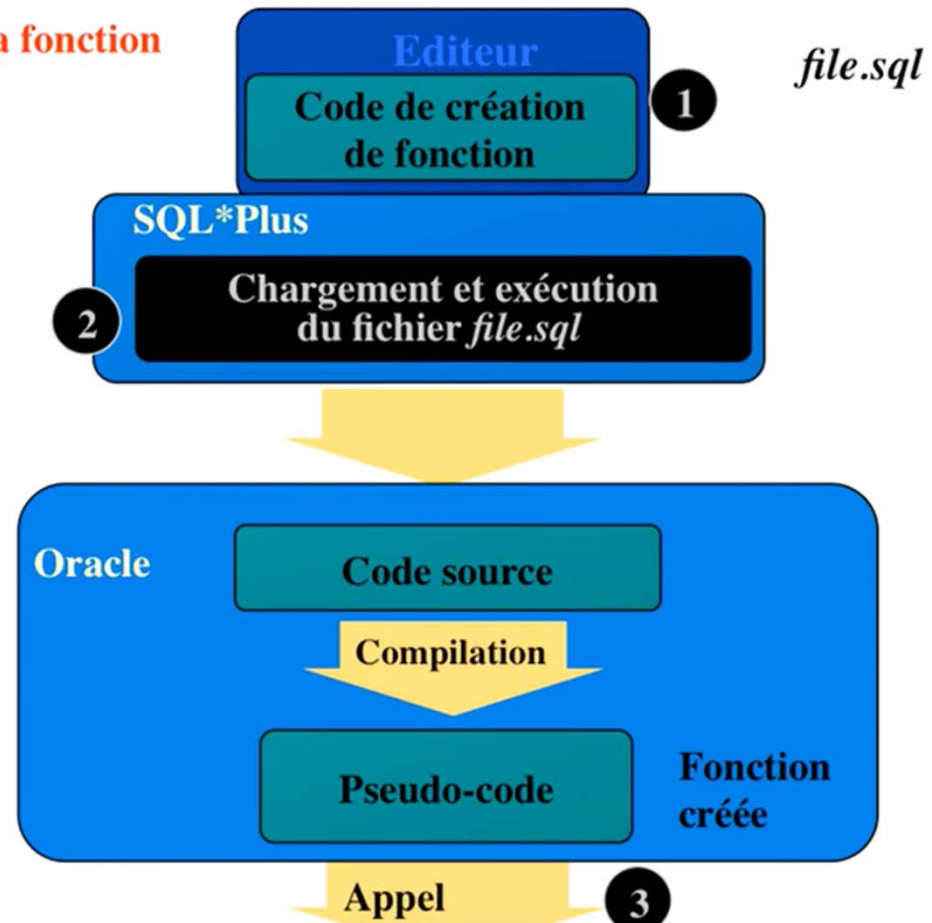
- ❖ Une fonction est un bloc PL/SQL nommé qui **renvoie une valeur**
- ❖ Une fonction peut être stockée en tant qu'objet de schéma dans la base de données en vue **d'exécutions répétées**
- ❖ Une fonction est **appelée dans une expression**

Syntaxe pour la création de fonctions

```
CREATE [OR REPLACE] FUNCTION function_name
  [(parameter1  datatype1,
   parameter2  datatype2,
   . . .)]
RETURN datatype -- ne doit pas inclure de spécification de taille
IS|AS
PL/SQL Block;
```

Le bloc PL/SQL doit comporter au moins une instruction **RETURN**.

Création de la fonction



Exemple de création d'une fonction stockée

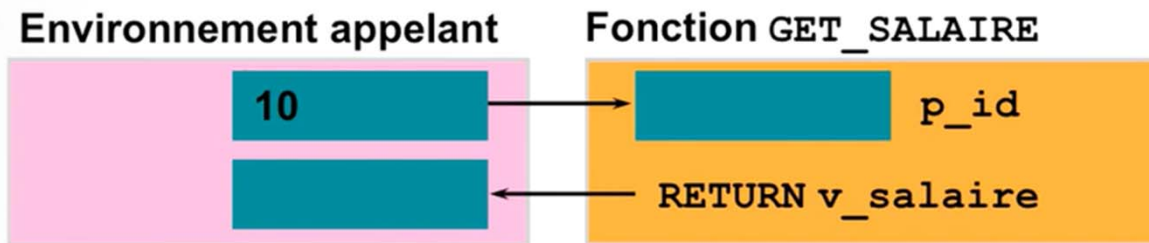
```
CREATE OR REPLACE FUNCTION get_salaire
    (p_id IN employees.employee_id%TYPE)
    RETURN NUMBER
IS
    v_salaire employees.salaire%TYPE :=0;
BEGIN
    SELECT salaire
    INTO    v_salaire
    FROM    employees
    WHERE   employee_id = p_id;
    RETURN v_salaire;
END get_salaire;
```


Exécuter des fonctions

- ❖ Appeler une fonction dans une expression PL/SQL
- ❖ Créer une variable destinée à recevoir la valeur renvoyée
- ❖ Exécuter la fonction.
- ❖ La valeur renvoyée par l'instruction RETURN sera placée dans la variable

RQ : Évitez d'utiliser les modes OUT et IN OUT avec les fonctions

Exemple d'exécution de fonctions



- 1 → Charger et exécuter le fichier get_salary.sql pour créer la fonction
- 2 →

```
VARIABLE salaire NUMBER
```
- 3 →

```
EXECUTE :salaire := get_salaire(10)
```
- 4 →

```
PRINT salaire
```

Avantages des fonctions définies par l'utilisateur dans les expressions SQL

- ❖ Elles complètent le langage SQL en permettant de réaliser des traitements qui seraient trop complexes, voire impossibles en SQL.
- ❖ Utilisées dans la clause WHERE pour filtrer les données, elles peuvent s'avérer plus efficaces qu'un filtrage au sein de l'application
- ❖ Elles permettent de manipuler les chaînes de caractères

Exemple d'appel de fonctions dans des expressions SQL

```
CREATE OR REPLACE FUNCTION tax(p_value IN NUMBER)
  RETURN NUMBER IS

BEGIN
  RETURN (p_value * 0.08);
END tax;
```

Appel fonction

```
SELECT employee_id, nom, salaire, tax(salaire)
FROM   employees WHERE department_id = 100;
```

Emplacements d'appel des fonctions définies par l'utilisateur

- ❖ Liste de sélection d'une commande **SELECT**
- ❖ Condition des clauses **WHERE** et **HAVING**
- ❖ Clauses **ORDER BY** et **GROUP BY**
- ❖ Clause **VALUES** de la commande **INSERT**
- ❖ Clause **SET** de la commande **UPDATE**

Exemple appel fonction :

```
SELECT employee_id, tax(salaire)
FROM employees
WHERE tax(salaire) > (SELECT tax(salaire)
                      FROM employees
                      WHERE department_id=30
                      )
ORDER BY tax(salaire) DESC;
```

Restrictions relatives à l'appel de fonctions à partir d'expressions SQL

Pour pouvoir être appelée depuis des expressions SQL, une fonction définie par l'utilisateur doit :

- ❖ être une **fonction stockée** (ce n'est pas le cas pour les procédures stockées.)
- ❖ Accepter uniquement des **paramètres IN**
- ❖ En tant que paramètres, accepter uniquement des **types de données SQL valides** (et non des types spécifiques au langage PL/SQL)
- ❖ **Renvoyer des types de données SQL** valides et non des types spécifiques au langage PL/SQL
- ❖ Les fonctions appelées depuis des expressions SQL (SELECT, UPDATE, DELETE en parallèle) **ne peuvent modifier** aucune table de la BDD

Supprimer une fonction stockée.

```
DROP FUNCTION function_name
```

Exemple :

```
DROP FUNCTION get_salaire;
```

Function dropped.

- Tous **les privilèges** accordés à une fonction sont annulés lorsque celle-ci est supprimée.
- Lorsque la syntaxe **CREATE OR REPLACE** est utilisée, la fonction est supprimée et recréée, mais dans ce cas, les privilèges accordés sur la fonction **ne sont pas affectés**.

Comparer les procédures et les fonctions

Procédures	Fonctions
S'exécutent en tant qu'instruction PL/SQL	Sont appelées dans une expression
Ne contiennent pas de clause RETURN dans l'en-tête	Doivent contenir une clause RETURN dans l'en-tête
Peuvent transférer zéro, une ou plusieurs valeurs	Doivent renvoyer une seule valeur
Peuvent contenir une instruction RETURN	Doivent contenir au moins une instruction RETURN

Synthèse

- ❖ Une fonction est **un bloc PL/SQL** nommé qui doit renvoyer une valeur
- ❖ La syntaxe **CREATE FUNCTION** permet de créer une fonction
- ❖ Une fonction est **appelée dans une expression**
- ❖ Une fonction stockée dans la base de données **peut être appelée dans des instructions SQL**
- ❖ La syntaxe **DROP FUNCTION** permet de **supprimer une fonction de la base de données**
- ❖ En règle générale, une procédure permet **d'exécuter une action** tandis qu'une fonction permet de **calculer une valeur**

Fonctions - Exemple

Exemple : Créer une fonction qui retourne le nom d'un employé.
employe (numemp, nomemp, salaire, date_embauche)

```
CREATE FUNCTION nom_employe ( numero IN NUMBER )  
                                RETURN VARCHAR2  
  
IS  
    nom employe.nomemp%type;  
BEGIN  
    SELECT nomemp into nom  
    FROM employe  
    WHERE numemp = numero ;  
    RETURN nom;  
END ;
```

Exécution et suppression

- Sous SQL*PLUS on exécute une procédure PL/SQL avec la commande **EXECUTE** :

EXECUTE nomProcédure(param1, ...);

Une fonction peut être utilisée dans une requête SQL

Exemple

```
select nome, sal, euro_to_dh(sal)
from emp;
```

- partir d'un bloc PL/SQL

```
DECLARE
    V_numP NUMBER := 14 ;
BEGIN
    augmentation_prix( V_numP, 0.1);
END ;
```

- Sous SQL PLUS:
 - 1) **EXECUTE** augmentation_prix(1, 0.2);
 - 2) VARIABLE nom varchar2(30) ;
EXECUTE :nom= nom_employe (2) ;
PRINT nom;

Type d'appel d'un sous programme

- L'appel d'un sous-programme (procédure ou fonction) peut être positionnel, mot clé (nommé) ou mixte.

- Paramètres positionnels

`augmentation_prix( 1, 0.2)`

- Paramètres mot clé

`augmentation_prix( Taux=>0.2, numerop=>1)`

- Mixtes

`augmentation_prix( 1, Taux=>0.2)`

- **Attention: Pour tous les appels mixtes, il faut que les notations positionnelles précèdent les notations nommées.**

Les packages

Les packages

- Un package (paquetage) est **l'encapsulation** d'objets de programmation PL/SQL dans une même unité logique de traitement tels : types, constantes, variables, procédures et fonctions, curseurs, exceptions.
- La création d'un package se fait en deux étapes:
 - Création des **spécifications** du package
 - spécifier à la fois les fonctions et procédures **publiques** ainsi que les déclarations des types, variables, constantes, exceptions et curseurs utilisés dans le **paquetage et visibles par le programme appelant**.
 - Création du **corps** du package
 - définit les procédures (fonctions), les curseurs et les exceptions qui sont déclarés dans les spécifications de la procédure. Cette partie peut définir d'autres objets de même type non déclarés dans les spécifications. Ces objets sont alors **privés**.
Cette partie **peut également contenir du code qui sera exécuté à chaque invocation du paquetage** par l'utilisateur (bloc d'initialisation).

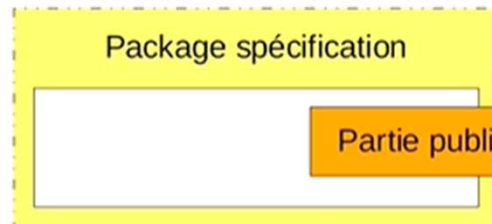
Présentation des packages

Les packages :

- ❖ Regroupent des types PL/SQL, des éléments et des sous-programmes présentant une relation logique
- ❖ Sont constitués de deux éléments :
 - Spécification (**déclaration**)
 - Corps (**définition**)
- ❖ Ne peuvent pas être appelés, paramétrés ou imbriqués
- ❖ Permettent au serveur Oracle de lire simultanément plusieurs objets en mémoire

Structure d'un package

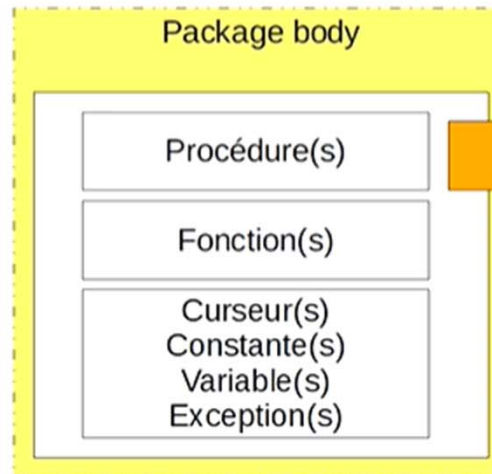
Un package est constitué de 2 composants



- Déclaration des éléments publics du package.
- Informations accessibles à tout utilisateur autorisé.

CREATE [OR REPLACE] PACKAGE ... ;

DROP PACKAGE [BODY] package_name;

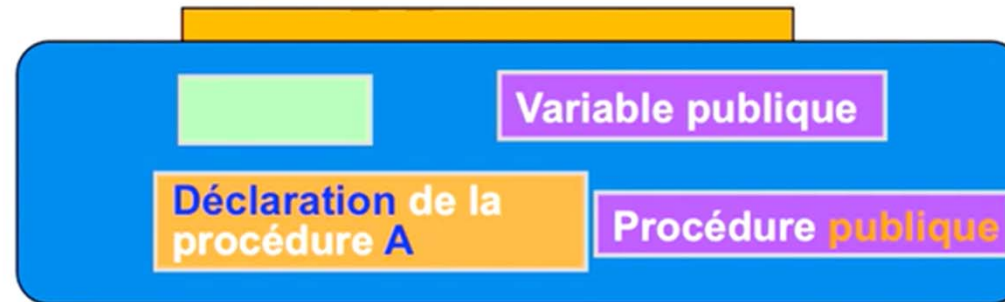


- Définition des éléments identifiés dans la partie « package spécification » .
- Définition de sous programmes (privés) utilisables seulement dans le package.
- Bloc d'initialisation

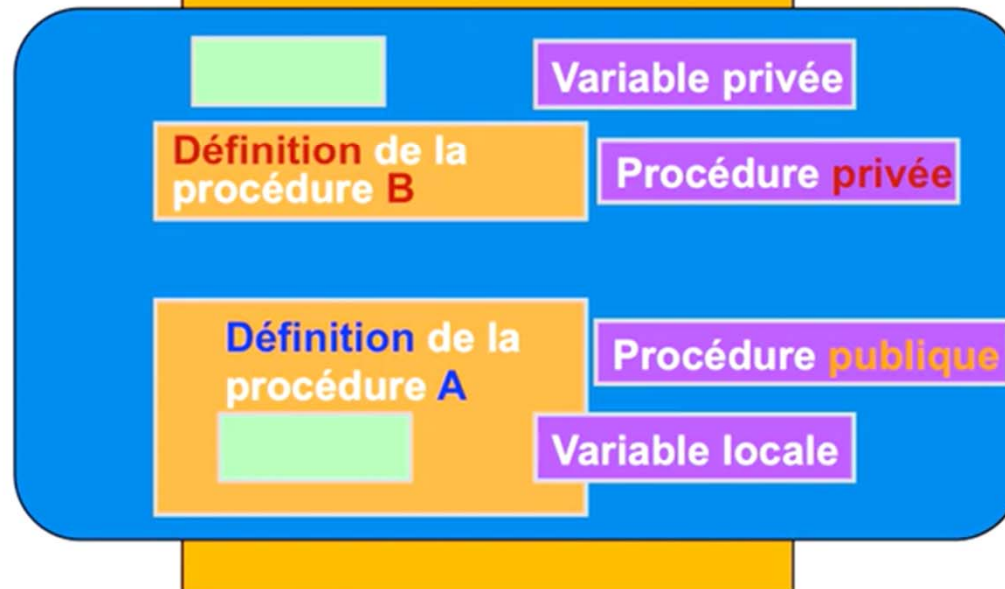
CREATE [or REPLACE] PACKAGE BODY ... ;

Composants d'un package

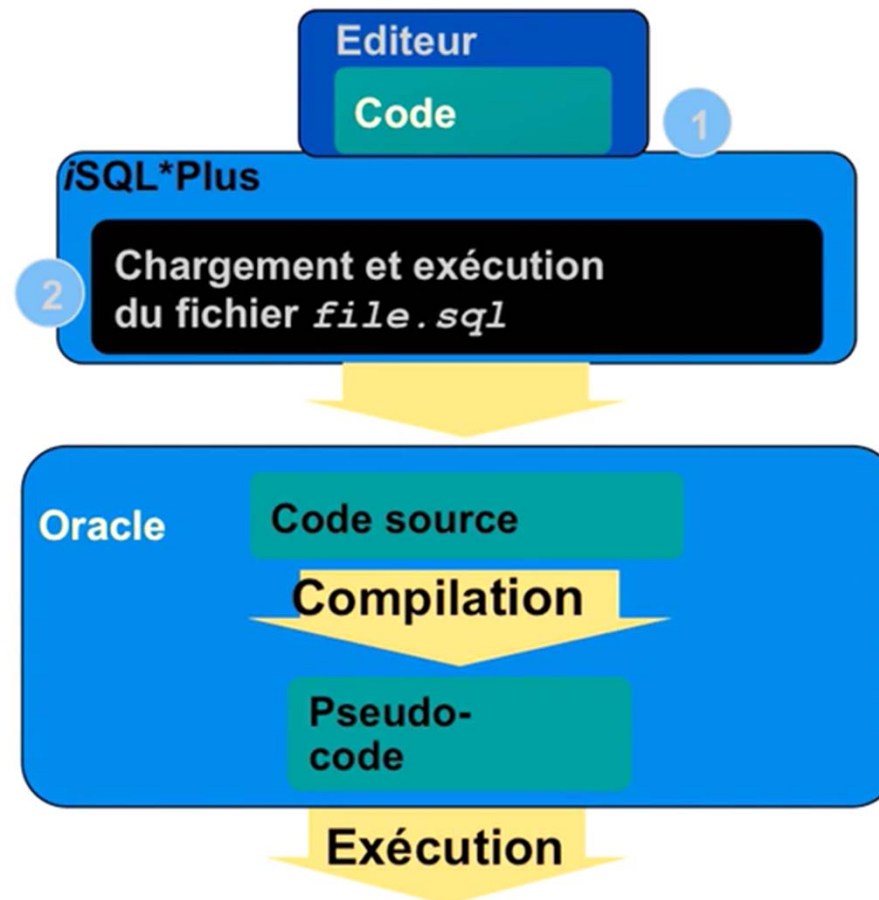
Spécification
du package



Corps du
package



Développer un package



Développer un package

- ❖ L'enregistrement du texte de l'instruction **CREATE PACKAGE** dans deux fichiers SQL distincts facilite les modifications ultérieures du package
 - Spécification : **CREATE PACKAGE**
 - Corps: **CREATE PACKAGE BODY**

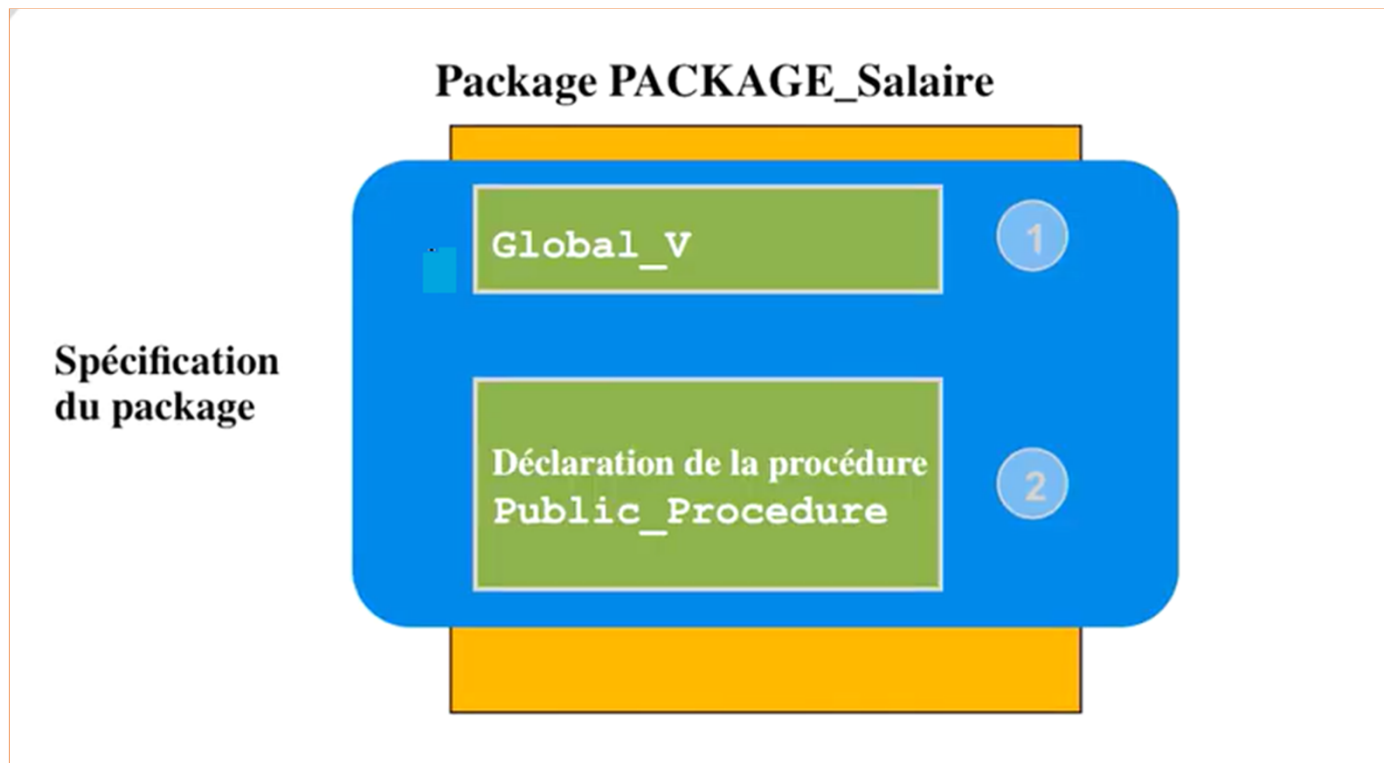
Créer la spécification du package

Syntaxe:

```
CREATE [OR REPLACE] PACKAGE package_name  
IS | AS  
    définitions des types utilisés dans le package;  
    Prototypes de toutes les procédures et fonctions du package;  
END package_name;
```

- ❖ L'option **REPLACE** supprime et recrée la spécification du package
- ❖ Toutes les structures déclarées dans une spécification de package peuvent être visibles par les utilisateurs disposant de privilèges sur le package

Les structures publiques



Les structures publiques

Exemple : spécifications package

```
CREATE OR REPLACE PACKAGE PACKAGE_Salaire
IS
    global_v NUMBER := 0.10; --initialisé à 0.10
    PROCEDURE Public_procedure(p_comm IN NUMBER);
END PACKAGE_Salaire;
/
```

Package created.

- g_var est une variable globale dont la valeur d'initialisation est 0,10.
- **Public_procedure** est une procédure publique implémentée dans le corps du package.

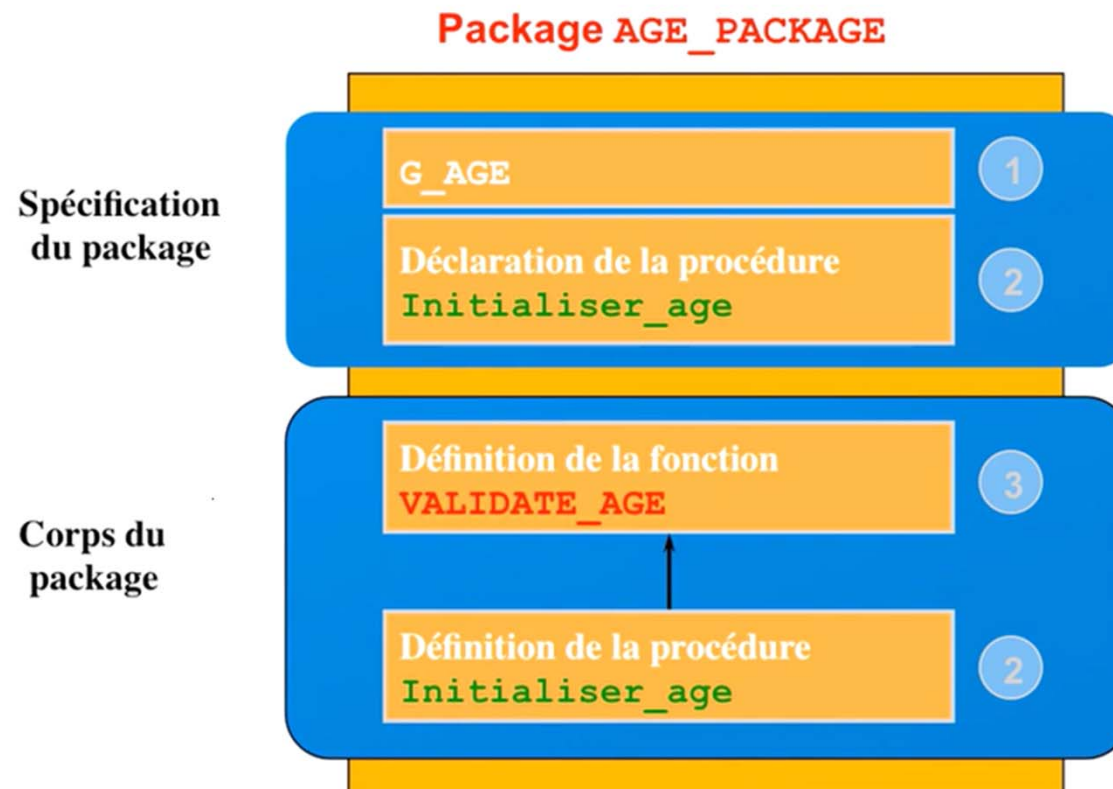
Le corps du package

Syntaxe:

```
CREATE [OR REPLACE] PACKAGE BODY package_name  
IS | AS  
    Déclaration de variables privées;  
  
END package_name;
```

- L'option **REPLACE** supprime et recrée le corps du package
- Les identificateurs définis exclusivement dans le corps du package sont des structures privées. Ils ne sont pas visibles à l'extérieur du corps du package
- Toutes les structures privées doivent être déclarées avant d'être utilisées dans les structures publiques

Structures publiques et privées



Création de corps de package

Exemple : age_pack.sql

```
CREATE OR REPLACE PACKAGE BODY AGE_PACKAGE
IS
    FUNCTION validate_age (Par_age IN NUMBER)
    RETURN BOOLEAN
    IS
        v_max_age    NUMBER;
    BEGIN
        SELECT      MAX(age)
        INTO        v_max_age
        FROM        client;
        IF    Par_age > v_max_age THEN RETURN(FALSE) ;
        ELSE    RETURN(TRUE) ;
        END IF;
    END validate_age;
...

```

Création de corps de package

age_pack.sql

```
PROCEDURE initialiser_age (p_age IN NUMBER)
IS
BEGIN
  IF validate_age(p_age) <
  THEN g_age:=p_age;
    DBMS_OUTPUT.PUT_LINE('Age n''est pas valide' );
  END IF;
END initialiser_age ;
END age_package;
/
```

Appel d'une fonction depuis une procédure du même package.

Structures publiques et privées

Spécification du package

```
Create or replace package test
Is
  Var1 Constant number:=10;
  Var2 varchar2(2):= 'ENSA';
  Procedure affichage;
End;
```

Corps du package

```
Create or replace package body test
Is
  Var3 varchar2(2):= 'FST';

  Procedure getVar
  Is
  Begin
    DBMS_OUTPUT.PUT_LINE(Var1 );
    DBMS_OUTPUT.PUT_LINE(Var3 );
  End;

  Procedure affichage
  Is
  Var4 varchar2(2):= 'EST';
  Begin
    getVar();
    DBMS_OUTPUT.PUT_LINE(Var2 );
    DBMS_OUTPUT.PUT_LINE(Var3 );
    DBMS_OUTPUT.PUT_LINE(Var4 );
  End;
End;
```

Appeler des structures de package

Appeler une procédure de package depuis SQL*Plus.

```
EXECUTE nom_package.nom_procedure(valeurs  
paramètres..)
```

Appeler une procédure de package dans un autre schéma.

```
EXECUTE scott. nom_package.nom_procedure (valeurs  
paramètres..)
```

Déclarer un package sans corps

```
CREATE OR REPLACE PACKAGE global_var IS
  PI CONSTANT  NUMBER  :=  3.14;
  AGE CONSTANT  NUMBER  :=  30;
  R CONSTANT  NUMBER  :=  10;
END global_var ;
/

EXECUTE DBMS_OUTPUT.PUT_LINE('age  = '|| global_var.age)
EXECUTE DBMS_OUTPUT.PUT_LINE('Air d'un disque  = '||
2*global_var.PI*global_var.R)
```

Référencer une variable publique depuis une procédure autonome

Exemple:

```
CREATE OR REPLACE PROCEDURE Air_Disque  
(air OUT NUMBER)  
IS  
BEGIN  
    air:= 2*global_var.PI*global_var.R;  
END;  
/
```

```
VARIABLE air NUMBER  
EXECUTE Air_Disque ( :air)  
PRINT air
```

Supprimer des packages

Utilisez la syntaxe suivante pour **supprimer** la **spécification** et le **corps** du package :

```
DROP PACKAGE package_name;
```

Utilisez la syntaxe suivante pour **supprimer** le **corps** du package :

```
DROP PACKAGE BODY package_name;
```


Supprimer des packages

Pour afficher les informations sur un package :

```
Select * from user_objects where object_name='nom package'
```

Pour afficher le code source d'une spécification

```
Select * from user_source  
Where name='nom package' and type ='PACKAGE'
```

Pour afficher le code source d'un Corps (Body)

```
Select * from user_source  
Where name='nom package' and type ='PACKAGE BODY'
```

Avantages liés aux packages

- ❖ **Modularité** : encapsule les structures associées
- ❖ **Conception simplifiée des applications** : la spécification et le corps sont codés et compilés séparément
- ❖ **Masquage d'informations** :
 - Seules les déclarations contenues dans la spécification du package sont **visibles et accessibles aux applications**
 - Les structures privées du corps du package sont **masquées et inaccessibles**
 - L'ensemble du code est **masqué dans le corps du package**

Avantages liés aux packages

❖ Performances accrues :

- **l'ensemble du package est chargé en mémoire** la première fois que celui-ci est référencé
- **une seule copie est chargée en mémoire** pour l'ensemble des utilisateurs

❖ **Surcharge** : plusieurs sous-programmes portant le même nom

Synthèse

- ❖ Optimiser l'organisation, la gestion, la sécurité et les performances en utilisant des packages
- ❖ Regrouper les procédures et les fonctions associées au sein d'un package
- ❖ Modifier un corps de package sans affecter sa spécification
- ❖ Définir un accès sécurisé à l'ensemble du package
- ❖ Masquer le code source aux utilisateurs
- ❖ Charger l'ensemble du package en mémoire au premier appel
- ❖ Réduire les accès au disque pour les appels ultérieurs.

Exercice

Soit le schéma relationnel suivant :

❖ **Client (Num_cl, Nom, prénom, age, ville)**

❖ **Ecrire un package contenant la spécification suivante:**

- ✓ Une procédure « Add_Client » pour insérer les informations d'un client
- ✓ Une procédure « Add_Client » pour insérer les informations d'un client sauf la ville
- ✓ Une procédure « get_Client » qui affiche les informations d'un client donné
- ✓ Une fonction « get_age » qui retourne l'age d'un client donné
- ✓ Une procédure « delete_client » qui supprime un client donné
- ✓ Une procédure « delete_client » pour supprimer les clients dont l'age est dans la liste { 20,30,40}

❖ **Implémenter le corps de toutes les spécifications:**

PL / SQL

Les exceptions

Gestion des erreurs

Le langage PL/SQL offre au développeur un mécanisme de **gestion des exceptions**. Il permet de préciser la logique du traitement des erreurs survenues dans un bloc PL/SQL. Il s'agit donc d'un point clé dans l'efficacité du langage qui permettra de **protéger l'intégrité du système**. Il existe deux types d'exception :

- ❑ **Interne** : Les exceptions internes sont générées par le moteur du système (division par zéro, connexion non établie, table inexistante, privilèges insuffisants, mémoire saturée, espace disque insuffisant, ...).
- ❑ **Externe** : Les exceptions externes sont générées par l'utilisateur .

Gestion des erreurs

Les erreurs ORACLE générées par le noyau sont numérotées (**ORA-xxxxx**). Il a donc fallu établir une **table de correspondance** entre les erreurs ORACLE et des noms d'exceptions.

Voici quelques exemples d'exceptions prédéfinis et des codes correspondants :

Nom d'exception	Erreur ORACLE	SQLCODE
CURSOR_ALREADY_OPEN	ORA-06511	-6511
DUP_VAL_ON_INDEX	ORA-00001	-1
INVALID_CURSOR	ORA-01001	-1001
INVALID_NUMBER	ORA-01722	-1722
LOGIN_DENIED	ORA-01017	-1017
NO_DATA_FOUND	ORA-01403	+100
PROGRAM_ERROR	ORA-06501	-6501
ROWTYPE_MISMATCH	ORA-06504	-6504
TIMEOUT_ON_RESOURCE	ORA-00051	-51
TOO_MANY_ROWS	ORA-01422	-1422
ZERO_DIVIDE	ORA-01476	-1476

Gestion des erreurs

Signification des erreurs Oracles présentées ci-dessus :

- **CURSOR_ALREADY_OPEN** : tentative d'ouverture d'un curseur déjà ouvert..
- **DUP_VAL_ON_INDEX** : violation de l'unicité lors d'une mise à jour détectée au niveau de l'index unique.
- **INVALID_CURSOR** : opération incorrecte sur un curseur, comme par exemple la fermeture d'un curseur qui n'a pas été ouvert.
- **INVALID_NUMBER** : échec de la conversion d'une chaîne de caractères en numérique.
- **LOGIN_DENIED** : connexion à la base échouée car le nom utilisateur ou le mot de passe est invalide.
- **NO_DATA_FOUND** : déclenché si la commande SELECT INTO ne retourne aucune ligne ou si on fait référence à un enregistrement non initialisé d'un tableau PL/SQL.
- **PROGRAM_ERROR** : problème général dû au PL/SQL.
- **ROWTYPE_MISMATCH** : survient lorsque une variable curseur d'un programme hôte retourne une valeur dans une variable curseur d'un bloc PL/SQL qui n'a pas le même type.
- **TIMEOUT_ON_RESOURCE** : dépassement du temps dans l'attente de libération des ressources (lié aux paramètres de la base).
- **TOO_MANY_ROWS** : la commande SELECT INTO retourne plus d'une ligne.
- **ZERO_DIVIDE** : tentative de division par zéro.

Exception utilisateur

DECLARE

...

nom_erreur EXCEPTION;

1

...

BEGIN

...

**IF (anomalie) THEN
RAISE nom_erreur;
END IF;**

2

...

EXCEPTION

WHEN nom_erreur THEN traitement;

3

Exception utilisateur

Exemple : Exception utilisateur

```
DECLARE
  Employe_age INT;
  invalid_age EXCEPTION;
BEGIN
  SELECT age INTO Employe_age FROM Employe WHERE nom='Salami';

  IF Employe_age <= 0 THEN
    RAISE invalid_age;
  ELSE
    DBMS_OUTPUT.PUT_LINE('L age de l'employé est: '||Employe_age);
  END IF;

  EXCEPTION
  WHEN invalid_age THEN dbms_output.put_line('L age est invalide');

END;
```

Exception Oracle - prédéfinie

Syntaxe :

```
BEGIN SELECT ... COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    statement1;
    statement2;
  WHEN TOO_MANY_ROWS THEN
    statement1;
  WHEN OTHERS THEN
    statement1;
    statement2;
    statement3;
END;
```

Exception Oracle - prédéfinie

Exemple : Exception prédéfinie Oracle

```
DECLARE
Var_Nom EMPLOYE.nom%TYPE;
BEGIN
SELECT nom INTO Var_Nom FROM EMPLOYE WHERE numéro=1;

EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('L''employé n''existe pas') ;

WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Erreur' ) ;
END;
/
```

Exception Oracle - Non prédéfinie

Dans ce type nous créons une **correspondance entre le code erreur ORACLE et le nom de l'exception**. Ce nom est choisi librement par le programmeur.

Pour affecter un nom à un code erreur, on doit utiliser une directive compilée (pragma) nommée **EXCEPTION_INIT**.

Syntaxe :

DECLARE

champ_obligatoire EXCEPTION;

1

PRAGMA EXCEPTION_INIT (champ_obligatoire, -1400);

2

BEGIN ...

EXCEPTION

WHEN champ_obligatoire THEN traitement;

3

END;

Exception Oracle - Non prédéfinie

Exemple : Exception Oracle – non prédéfinie

```
DECLARE  
Erreur_data EXCEPTION;  
Var_Nom EMPLOYE.nom%TYPE;  
PRAGMA EXCEPTION_INIT (Erreur_data , -100);  
BEGIN  
SELECT nom INTO var_nom FROM Employé WHERE code=1;  
EXCEPTION  
    WHEN Erreur_data THEN  
        DBMS_OUTPUT.PUT_LINE('L'employé n'existe pas' ) ;  
END;
```

Le code erreur -100 correspond aux données inexistantes .

Les déclencheurs/ Triggers

Objectifs

- ❖ Décrire différents types de déclencheur / Trigger
- ❖ Décrire les déclencheurs de base de données et leur utilisation
- ❖ Créer des déclencheurs de base de données
- ❖ Décrire les règles d'activation des déclencheurs de base de données
- ❖ Supprimer des déclencheurs de base de données

Déclencheur / Trigger

Un déclencheur :

- ❖ est une procédure ou un bloc PL/SQL associé à la base de données, à une table, à une vue ou à un schéma
- ❖ S'exécute de façon implicite lorsqu'un événement donné se produit
- ❖ il peut s'agir d'un :
 - **déclencheur applicatif**, qui s'exécute lorsqu'un événement se produit dans une application donnée
 - **déclencheur de base de données**, qui s'exécute lorsqu'un événement de type données (LMD) ou système (connexion ou arrêt) se produit dans un schéma ou une base de données

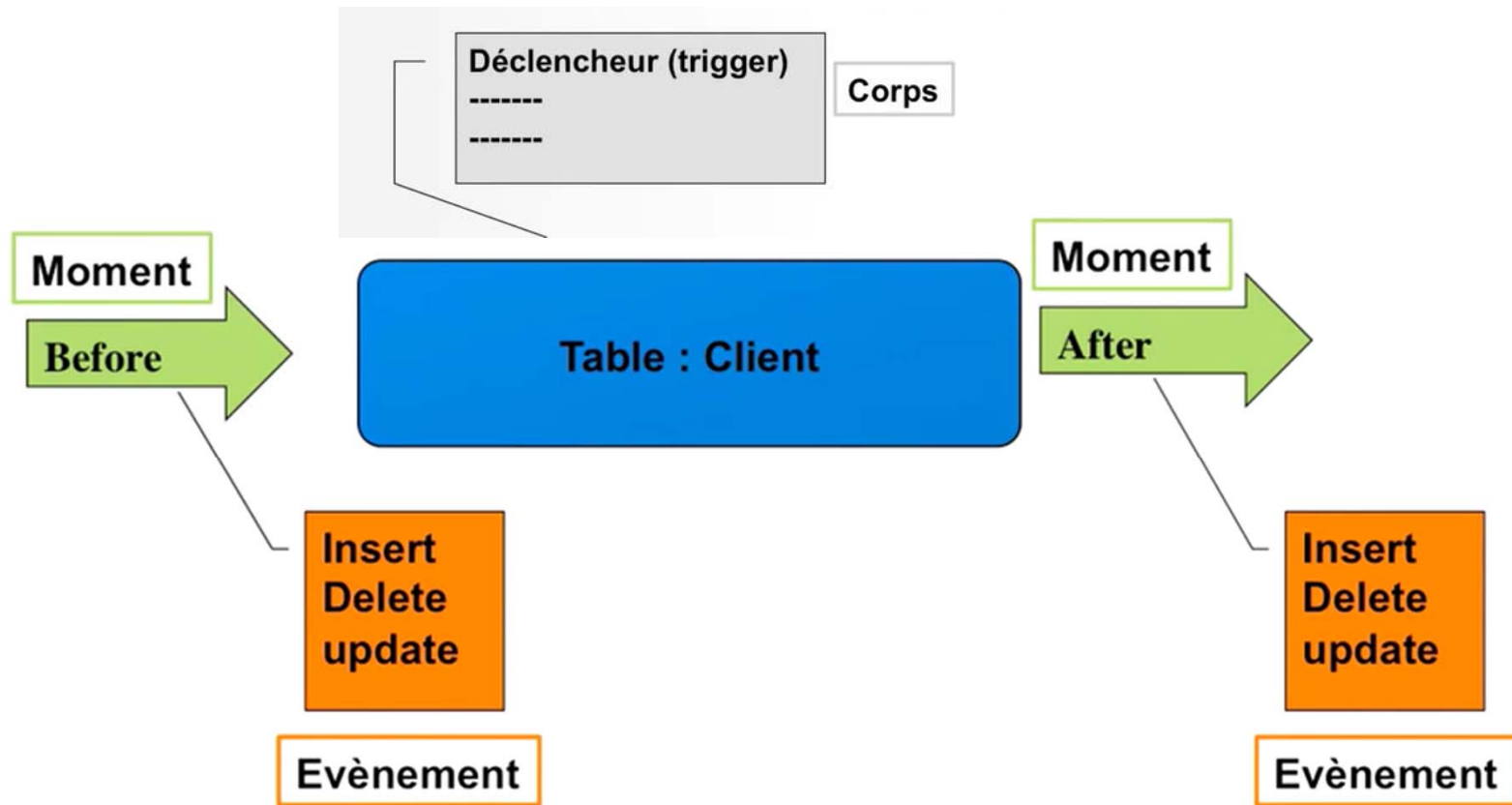
Règles relatives à la conception de déclencheurs

- ❖ Il est conseillé de concevoir des déclencheurs pour :
 - **exécuter des actions associées**
 - **centraliser des opérations globales**
- ❖ Si le code PL/SQL est très long, **créer des procédures stockées** et les appeler dans un déclencheur
- ❖ L'utilisation excessive de déclencheurs peut entraîner des interdépendances complexes dont la gestion peut s'avérer difficile dans les applications volumineuses

Utilisation des déclencheurs

- ❖ Sécurité ou contrôle d'accès : (exp: rendre les données disponibles uniquement dans les horaires de travail)
- ❖ Audit ou logging : (exp: sauvegarder l'historique des opérations effectué par les utilisateurs)
- ❖ Réplication des données

Les éléments d'un déclencheur



Créer des déclencheurs LMD

Éléments

Une instruction de déclenchement comporte les éléments suivants :

1. Moment du déclenchement
 - **BEFORE** (Avant), **AFTER** (Après)
2. Événement déclencheur : **INSERT**, **UPDATE** ou **DELETE**
3. Nom de la table : sur la table ou la vue
4. Type de déclencheur : ligne ou instruction
5. Clause **WHEN** : condition restrictive par ligne
6. Corps du déclencheur : bloc PL/SQL

Composants des déclencheurs LMD

1. Moment

Moment du déclenchement : à quel moment le déclencheur doit-il s'exécuter ?

- ❖ **BEFORE** : exécution du corps du déclencheur avant le déclenchement de l'événement LMD sur une table
- ❖ **AFTER** : exécution du corps du déclencheur après le déclenchement de l'événement LMD sur une table

Composants des déclencheurs LMD

2. Événement

Événement utilisateur déclencheur : quelle instruction LMD entraîne l'exécution du déclencheur ?

Vous pouvez utiliser les instructions suivantes :

- ❖ INSERT
- ❖ UPDATE
- ❖ DELETE

Composants des déclencheurs LMD

3. Type

Type de déclencheur : le corps du déclencheur doit-il s'exécuter une seule fois ou pour chaque ligne concernée par l'instruction ?

- ❖ **Instruction** : le corps du déclencheur s'exécute une seule fois pour l'événement déclencheur. Il s'agit du comportement **par défaut**. Un déclencheur sur instruction s'exécute une fois, même si aucune ligne n'est affectée.
- ❖ **Ligne** : le corps du déclencheur s'exécute une fois pour chaque ligne concernée par l'événement déclencheur. Un déclencheur sur ligne ne s'exécute pas si l'événement déclencheur n'affecte aucune ligne

Composants des déclencheurs LMD

4. Corps

Corps du déclencheur : quelle action le déclencheur doit-il effectuer ?

Le corps du déclencheur est un bloc PL/SQL ou un appel de procédure (PL/SQL ou Java).

RQ

- Les déclencheurs sur ligne utilisent des noms de corrélation pour accéder aux anciennes ou nouvelles valeurs de colonne de la ligne en cours de traitement.
- La taille d'un déclencheur est limitée à **32 Ko**.

Séquence d'exécution

Manipulation concerne une seule ligne,

Instruction LMD

```
INSERT INTO departments (department_id,  
                        department_name, location_id)  
VALUES (400, 'CONSULTING', 2400);
```

Action de déclenchement

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
20	Marketing	1800
30	Purchasing	1700
...		
400	CONSULTING	2400

→ Déclencheur sur
instruction BEFORE

→ Déclencheur sur ligne BEFORE

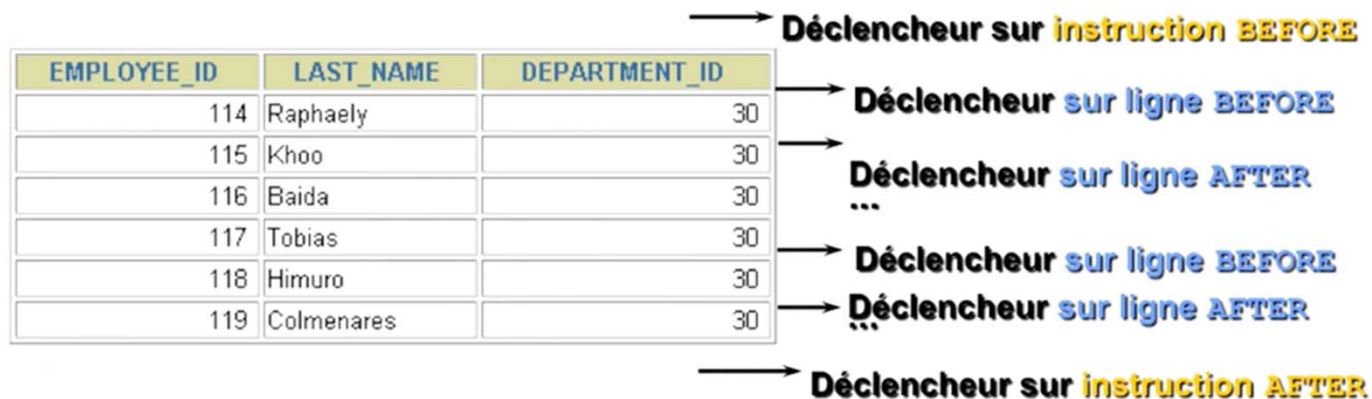
→ Déclencheur sur ligne AFTER

→ Déclencheur sur instruction
AFTER

Séquence d'exécution

Manipulation concerne plusieurs lignes,

```
UPDATE employees
  SET salary = salary * 1.1
 WHERE department_id = 30;
```



Création de déclencheurs sur une instruction LMD

Syntaxe :

```
CREATE [OR REPLACE] TRIGGER trigger_name
  timing - - BEFORE ou AFTER
  event1 [OR event2 OR event3] - - INSERT, UPDATE, DELETE
  ON table_name
  trigger_body
```

RQ: Les noms des déclencheurs doivent être uniques au sein d'un même schéma

Création de déclencheurs sur une instruction LMD

Exemple :

```
CREATE OR REPLACE TRIGGER secure_client
BEFORE INSERT OR DELETE ON client
BEGIN
  IF (TO_CHAR(SYSDATE,'DY') IN ('SAT','SUN')) OR
    (TO_CHAR(SYSDATE,'HH24:MI')
      NOT BETWEEN '08:00' AND '18:00')
  THEN
    Raise_application_error(-20011,' Vous ne pouvez pas faire une mise
    à jour d'un client dans ce temps');
  END IF;
END;
```

RQ : En cas d'échec d'un déclencheur de base de données, l'instruction de déclenchement est annulée.

Prédicats conditionnels

```
CREATE OR REPLACE TRIGGER secure_client
BEFORE INSERT OR DELETE OR UPDATE ON client
BEGIN
  IF (TO_CHAR(SYSDATE,'DY') IN ('SAT','SUN')) OR
     (TO_CHAR(SYSDATE,'HH24:MI') NOT BETWEEN '08:00' AND '18:00')
  THEN
    IF DELETING THEN
      Raise_application_error(-20011, ' Vous ne pouvez pas supprimer un client dans ce
temps');
    ELSIF INSERTING THEN
      Raise_application_error(-20012, ' Vous ne pouvez pas insérer un client dans ce
temps');
    ELSIF UPDATING ('nom') THEN
      Raise_application_error(-20013, Vous ne pouvez pas modifier le nom d'un client dans
ce temps');
    ELSE
      dbms_output.put_line(' Vous pouvez modifier la table client seulement dans l'horaire
normal ');
    END IF;
  END IF;
END;
```


Créer un déclencheur sur ligne LMD

Syntaxe :

```
CREATE [OR REPLACE] TRIGGER trigger_name
    timing
    event1 [OR event2 OR event3]
    ON table_name
    [REFERENCING OLD AS old | NEW AS new]
FOR EACH ROW
    [WHEN (condition)]
    trigger_body
```

Utilisation de :NEW et :OLD

- ▶ Si nous ajoutons un client dont le nom est toto alors nous récupérons ce nom grâce à la variable :NEW.nom
- ▶ Dans le cas de suppression ou modification, les anciennes valeurs sont dans la variable :OLD.nom

Créer un déclencheur sur ligne LMD

| Exemple :

```
CREATE OR REPLACE TRIGGER restrict_salaire
BEFORE UPDATE OF salaire ON employees
FOR EACH ROW
BEGIN
    IF :NEW.salaire > 15000
    THEN
        Raise_application_error (-20001, ' Vous ne pouvez pas modifier le
salaire de cet employé ');
    END IF;
END;
```

Trigger created.

```
UPDATE employees
SET salaire = 15500
WHERE nom = 'Essoufi';
```

Vous ne pouvez pas modifier le salaire de cet employé

Utiliser les qualificatifs OLD et NEW

```
CREATE OR REPLACE TRIGGER audit_emp_values
  AFTER DELETE OR INSERT OR UPDATE ON employees
  FOR EACH ROW
BEGIN
  INSERT INTO audit_emp_table (user_name, times_user,
    id, Ancien_nom, nouveau_nom, Ancienne_fonction,
    nouvelle_fonction, Ancien_salaire, nouveau_salaire)
  VALUES (USER, SYSDATE, :OLD.employee_id,
    :OLD.nom, :NEW.nom, :OLD.fonction,
    :NEW.fonction, :OLD.salaire, :NEW.salaire );
END;
```

Restreindre l'action d'un déclencheur sur ligne

```
CREATE OR REPLACE TRIGGER emp_salaire
  BEFORE INSERT OR UPDATE OF salaire ON employees
  FOR EACH ROW
  WHEN (NEW.numero_employe = 1)
BEGIN

  IF UPDATING ('salaire') THEN
    IF :NEW.salaire < :OLD.salaire THEN
      raise_application_error (-20001, 'Attention,
Diminution De salaire ');
    END IF;
  END IF;
END;
```

Différences entre les déclencheurs et les procédures

Déclencheurs	Procédures
Définis via la commande CREATE TRIGGER Le dictionnaire de données contient le code source dans <u>USER_TRIGGERS</u> Appel implicite Les instructions COMMIT, et ROLLBACK ne sont pas autorisées	Définis via la commande CREATE PROCEDURE Le dictionnaire de données contient le code source dans <u>USER_SOURCE</u> Appel explicite Les instructions COMMIT, et ROLLBACK sont autorisées

Gérer les déclencheurs

Désactiver ou réactiver un déclencheur de base de données :

```
ALTER TRIGGER trigger_name DISABLE | ENABLE
```

Désactiver ou réactiver tous les déclencheurs d'une table :

```
ALTER TABLE table_name DISABLE | ENABLE ALL TRIGGERS
```

Supprimer un déclencheur

Pour supprimer un déclencheur de la base de données, utiliser la syntaxe DROP TRIGGER :

```
DROP TRIGGER trigger_name;
```

Exemple :

```
DROP TRIGGER secure_emp;
```

Remarque : Lorsqu'une table est supprimée, tous ses déclencheurs sont également supprimés

Fin Elément PL / SQL